

Data Similarity and Clustering

Data Science

COMP5122M

Lecturer: Duygu Sarikaya

What will you learn?

- Calculating Data Similarity
- Data Distance Metrics
- k-Means Clustering
- Agglomerative Clustering
- Hierarchical clustering and dendrograms

What is Data Mining?

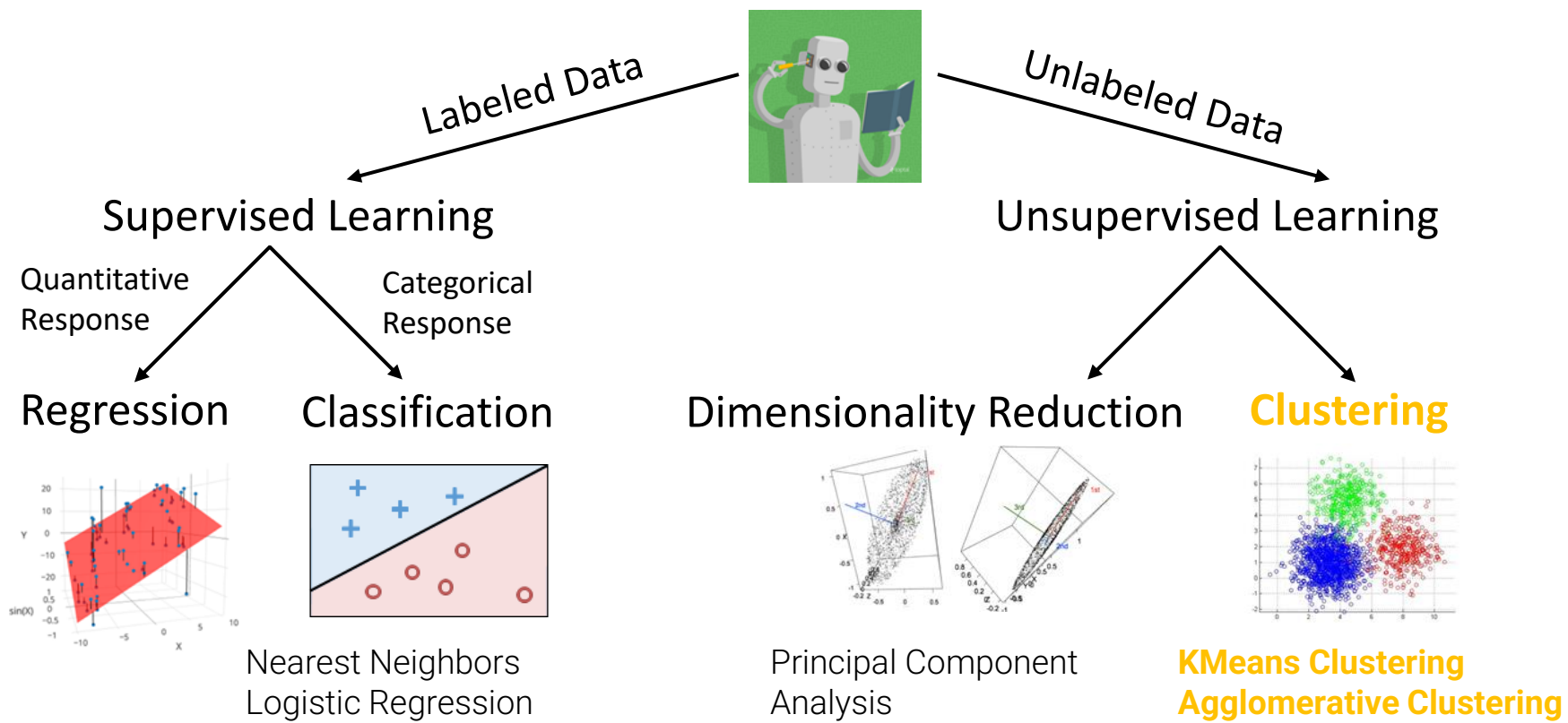
- *Data mining, also known as knowledge discovery in data (KDD), is the process of uncovering patterns and other valuable information from large data sets.*

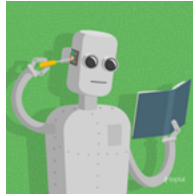
Data Mining vs Machine Learning

- *Data mining looks for patterns that already exist in the data, while machine learning learns from past data to predict future outcomes.*

	Data Mining	Machine Learning
Focus	Discovery of hidden patterns or knowledge from data	Development of algorithms that learn from data
Goal	Extract insights and information from existing datasets	Build models to make predictions or perform tasks
Usage	Identifying patterns, trends, and anomalies	Predictive modeling, classification, clustering, etc.

(sources: <https://www.simplilearn.com/data-mining-vs-machine-learning-article>)





Labeled Data

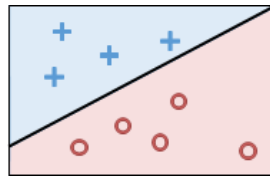
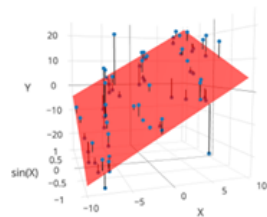
Supervised Learning

Quantitative
Response

Categorical
Response

Regression

Classification



In “Supervised Learning”:

- Goal is to create a function that maps inputs to outputs.
- Model is learned from example input/output pairs. Each pair consists of:
 - Input vector.
 - Output value (**label**).
- Regression: Output value is quantitative.
- Classification: Output value is categorical.

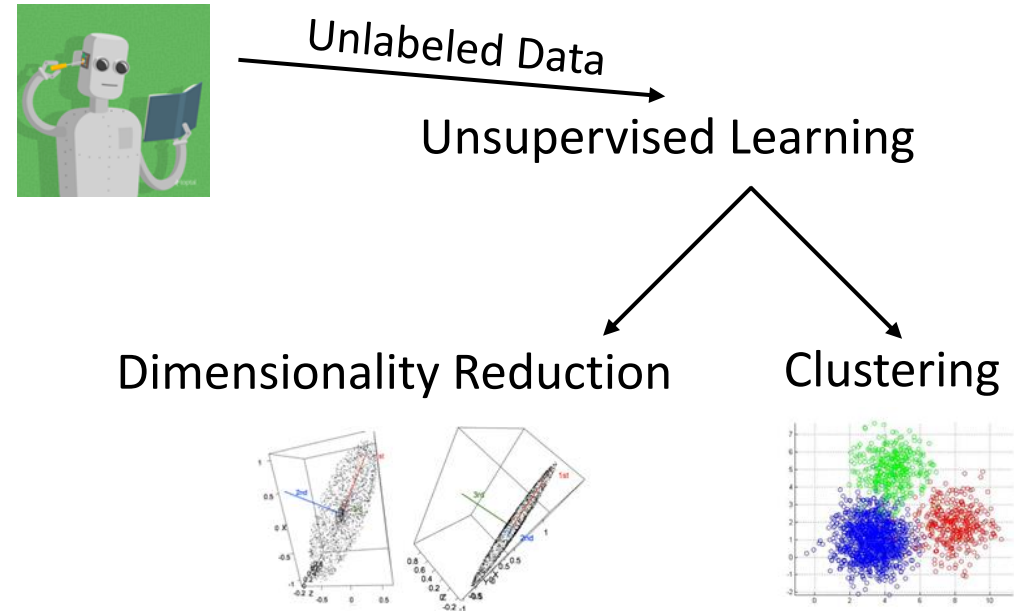
In “Unsupervised Learning”:

- Goal is to identify patterns in **unlabeled** data.
 - We do not have input/output pairs.
 - Note: Sometimes we may have labels, but we’re just choosing to ignore them (e.g. PCA on labeled data).

Problem: Dimensionality Reduction.

- Solution: PCA.

Problem: Clustering.



Unsupervised Learning

- Unsupervised learning includes all kinds of machine learning where there is no known output, no teacher to instruct the learning algorithm. -> no supervision
- In unsupervised learning, the learning algorithm is just shown the input data and asked to extract knowledge from this data.

Transformations of the dataset

- **Unsupervised transformations** of a dataset are algorithms that **create a new representation of the data** which might be easier for humans or other machine learning algorithms to understand compared to the original representation of the data.

For example,

Dimensionality Reduction takes a high-dimensional representation of the data, consisting of many features, and finds a new way to represent this data that summarizes the essential characteristics with fewer features.

- Dimensionality reduction to two dimensions for visualization purposes
- Topic extraction on collections of text documents

Clustering

- Clustering algorithms, on the other hand, partition data into distinct groups of similar items.
- The goal is to split up the data in such a way that points within a single cluster are very similar and points in different clusters are different.
- Similarly to classification algorithms, clustering algorithms assign (or predict) a cluster to a particular data point.

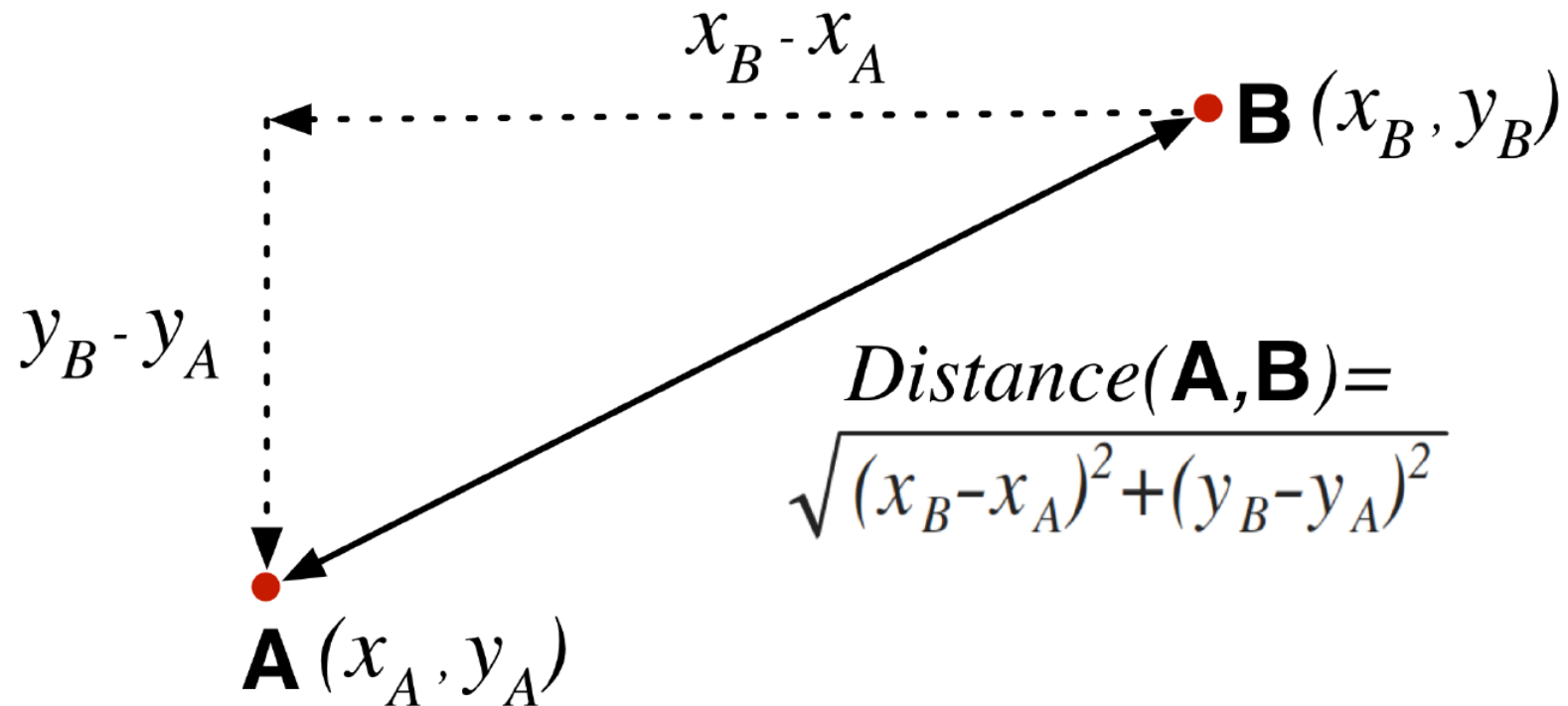
For example: Organizing uploaded pictures, group together pictures that show the same person or location.

Data Similarity

- Once an object can be represented as data, we can measure the similarity between objects, or alternatively the distance between objects.
- Let's consider we represent each object as a feature vector. Then, the closer two objects are in the space defined by the features, the more similar they are.
- • Amazon, Netflix, Spotify use similarity to provide recommendations
“People who like X also like Y”

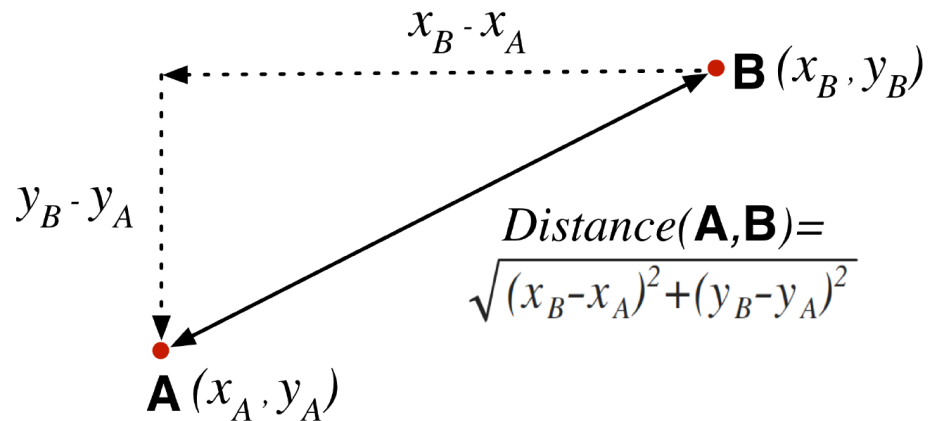
Data Similarity: Euclidian Distance

Attribute	Person A	Person B
Age	23	40
Years at current address	2	10
Residential status (1=Owner, 2=Renter, 3=Other)	2	1



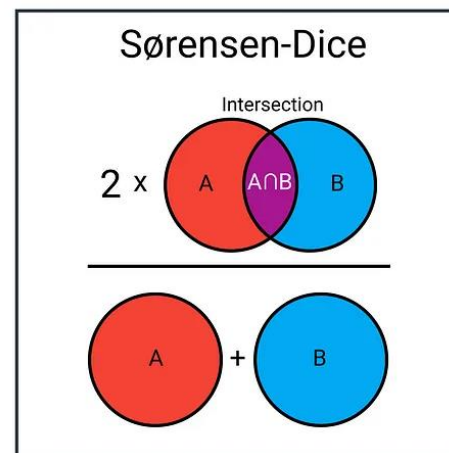
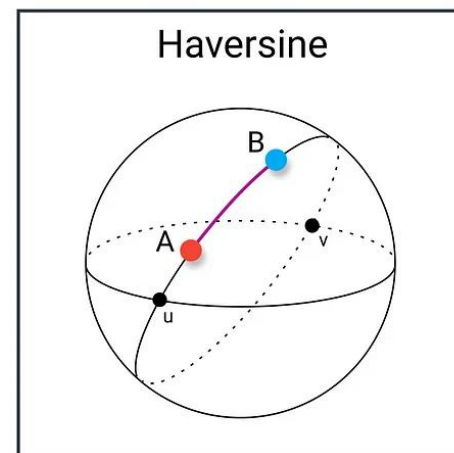
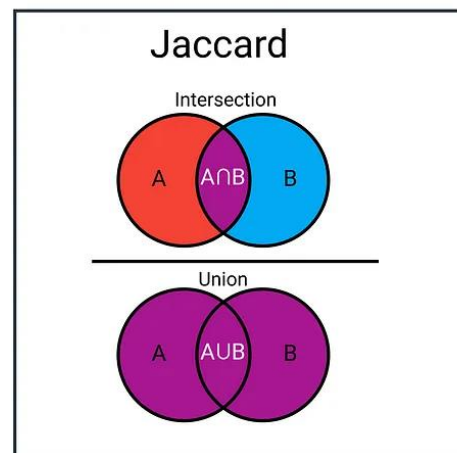
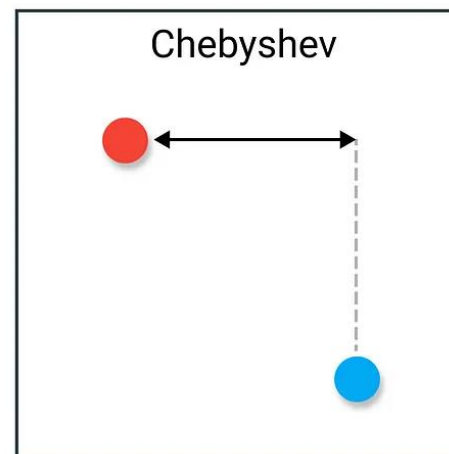
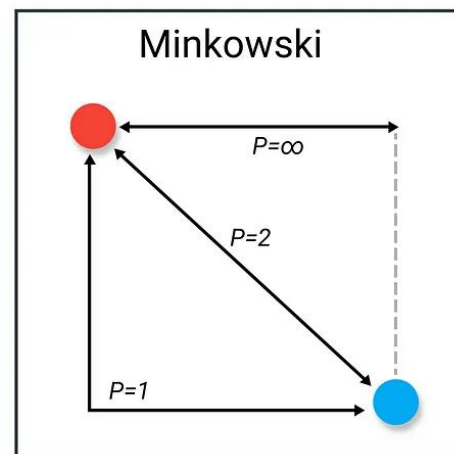
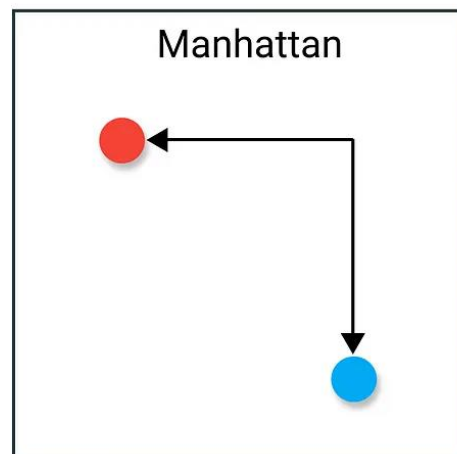
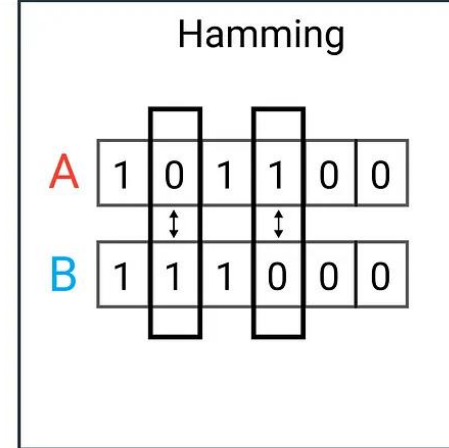
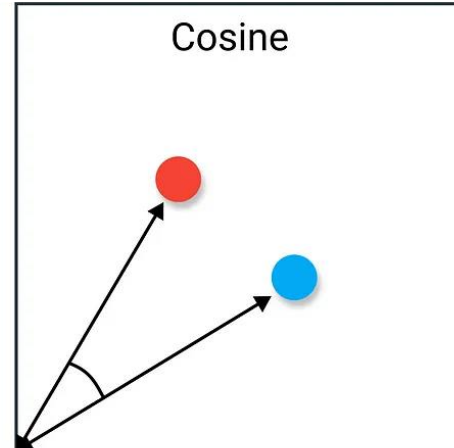
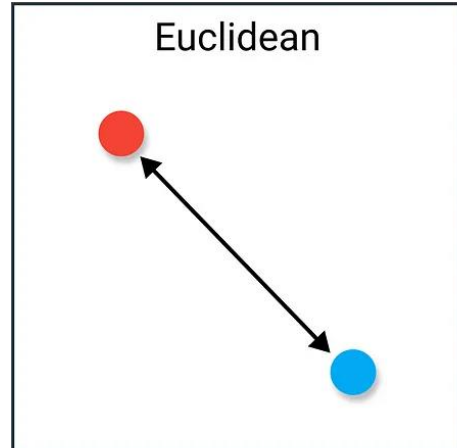
Data Similarity: Euclidian Distance

Attribute	Person A	Person B
Age	23	40
Years at current address	2	10
Residential status (1=Owner, 2=Renter, 3=Other)	2	1



- What if there are more variables?

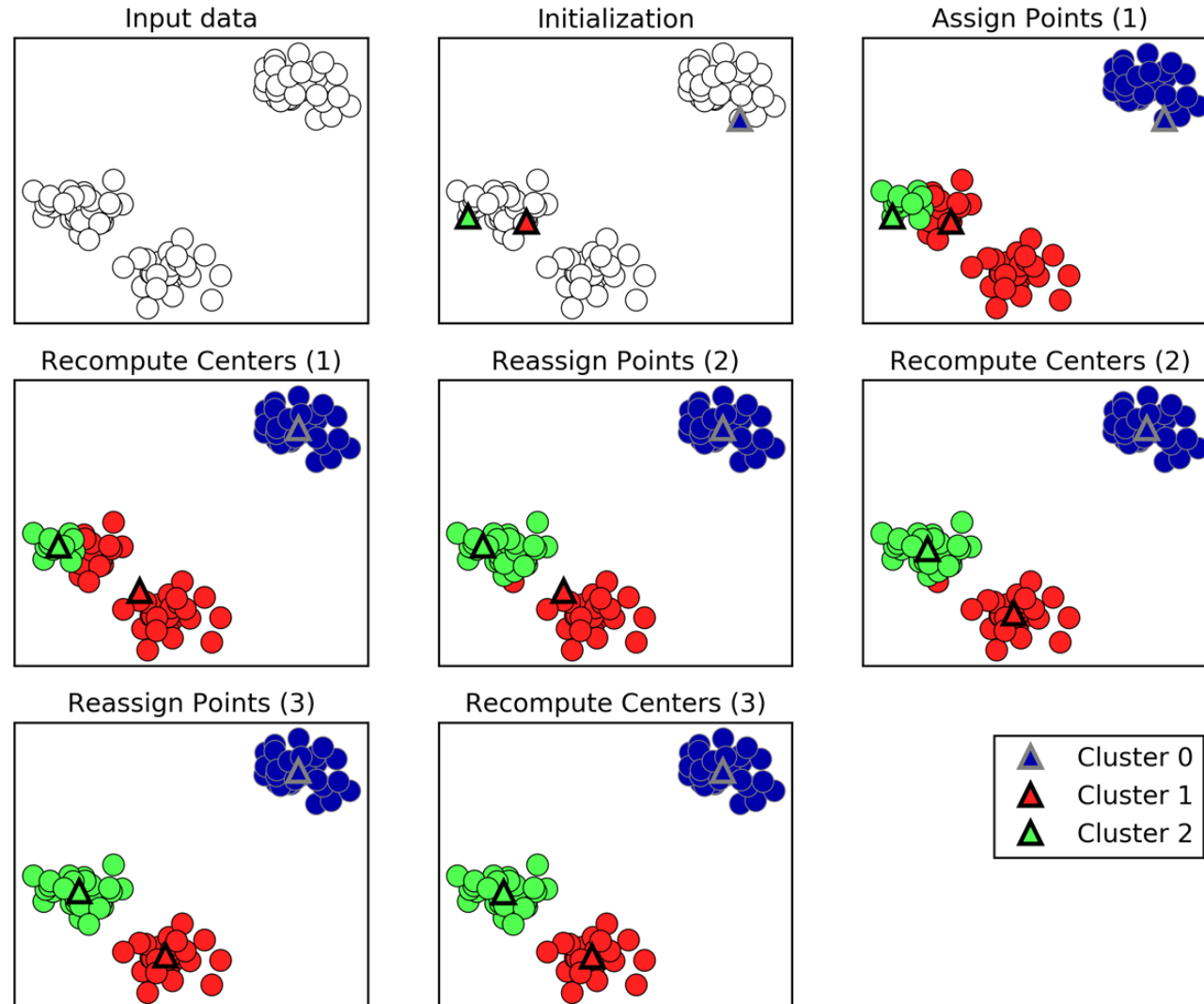
$$\sqrt{(d_{1,A} - d_{1,B})^2 + (d_{2,A} - d_{2,B})^2 + \dots + (d_{n,A} - d_{n,B})^2}$$



k-Means Clustering

- k-means clustering is one of the simplest and most commonly used clustering algorithms.
- It tries to find cluster centers that are representative of certain regions of the data.
- The algorithm alternates between two steps: assigning each data point to the closest cluster center, and then setting each cluster center as the mean of the data points that are assigned to it.
- The algorithm is finished when the assignment of instances to clusters no longer changes.

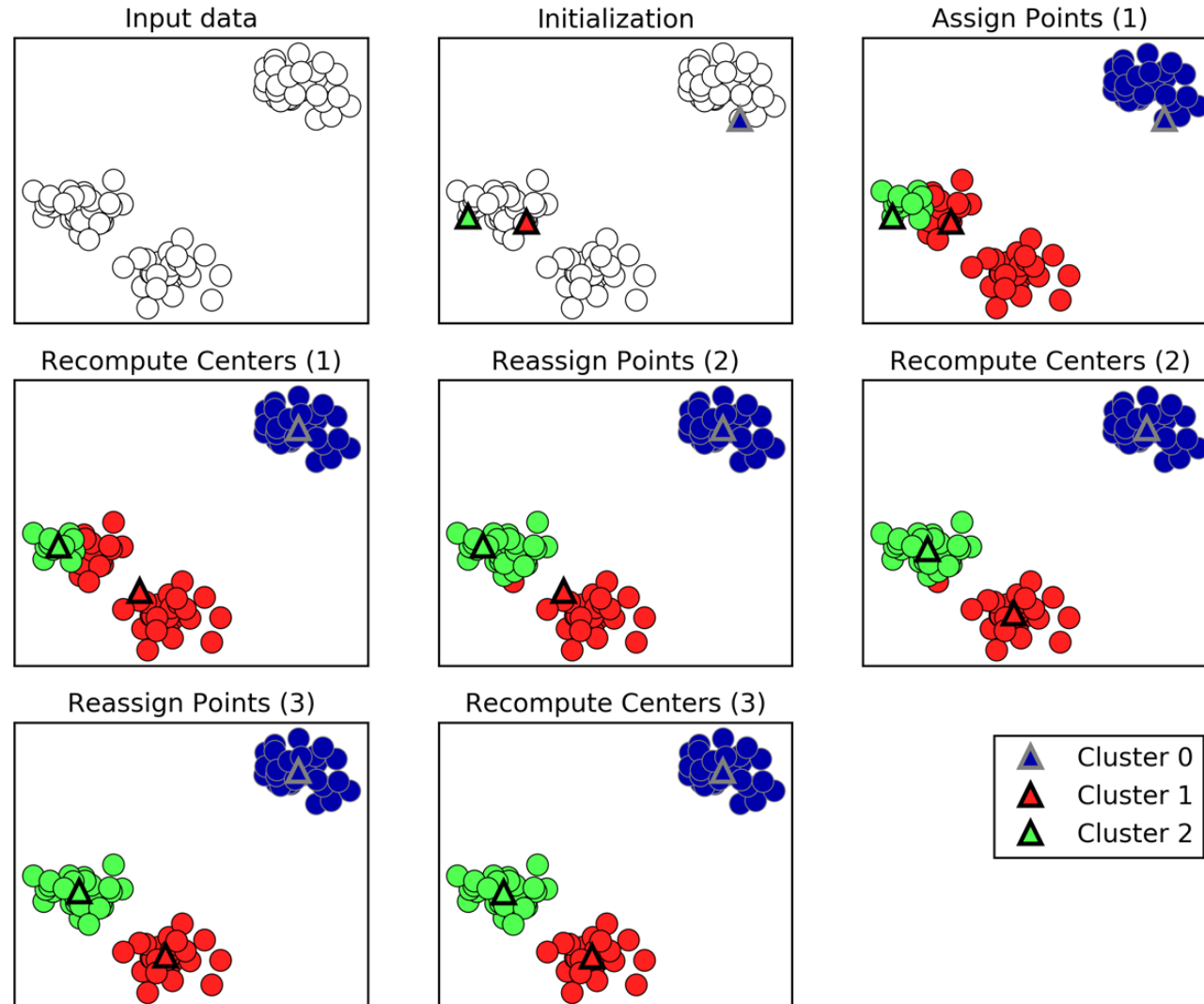
k-Means Clustering



Cluster centers are shown as triangles, while data points are shown as circles.

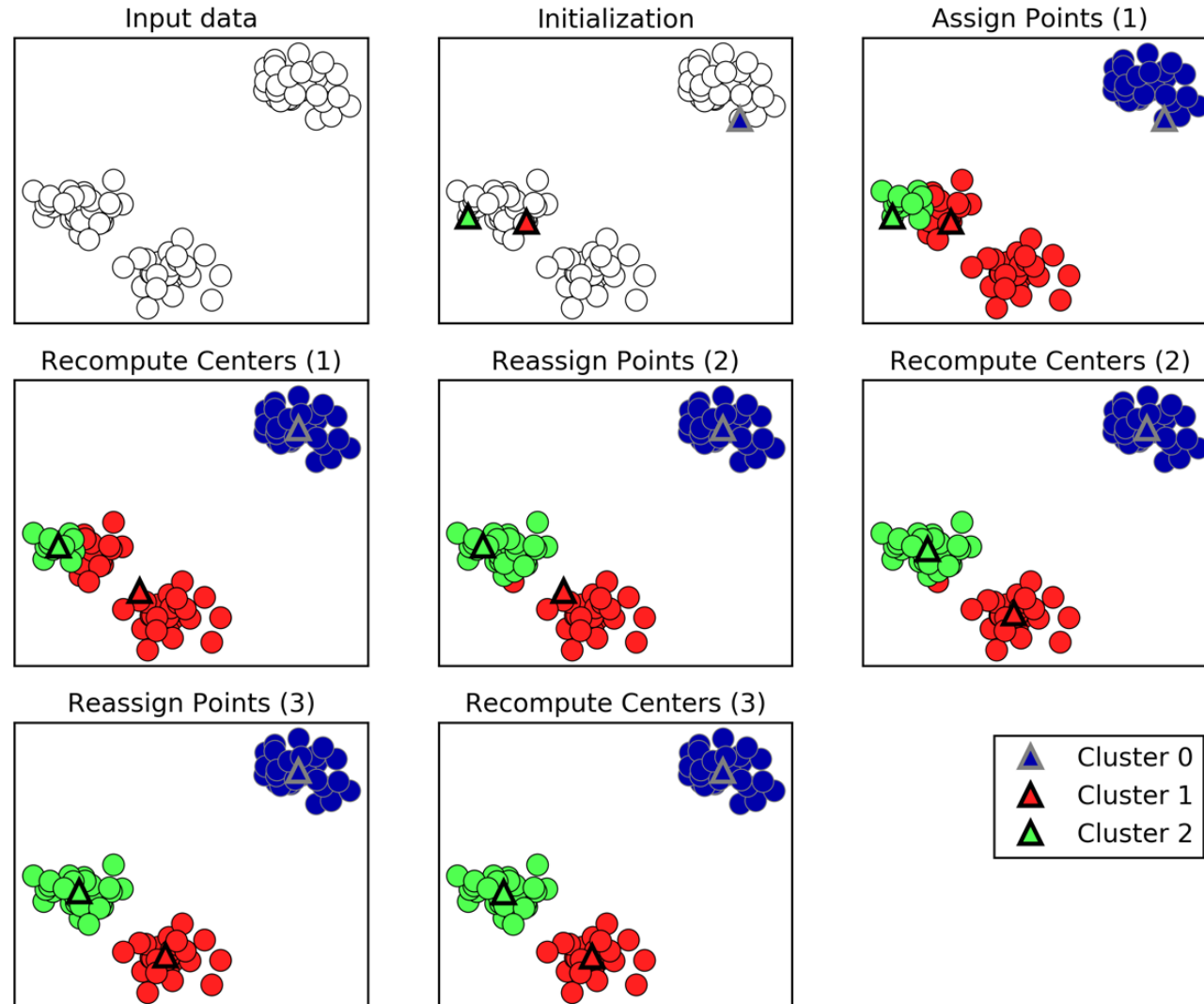
Colors indicate cluster membership.

k-Means Clustering



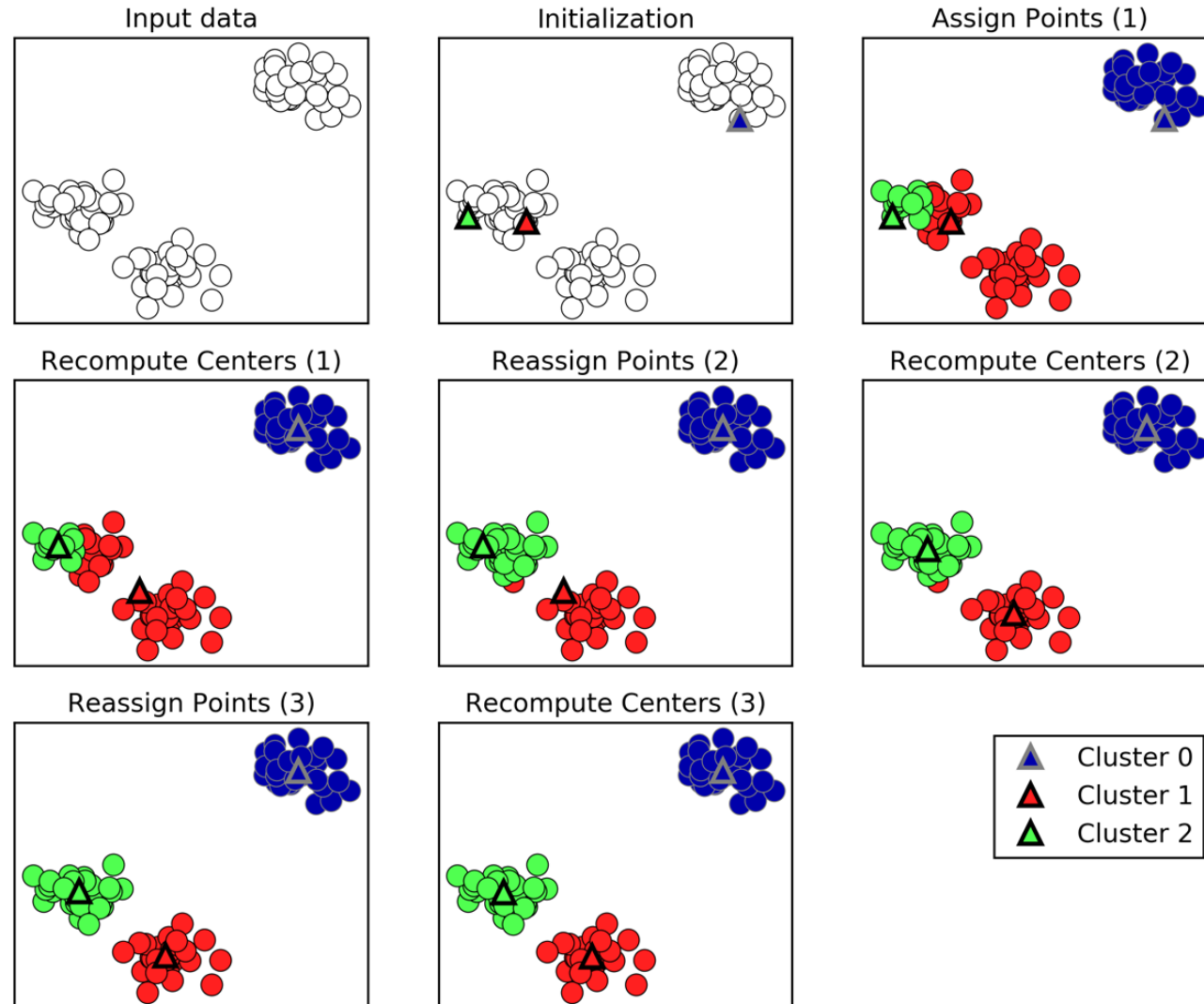
We are looking for three clusters, so the algorithm was initialized by declaring three data points randomly as cluster centers (“Initialization”).

k-Means Clustering



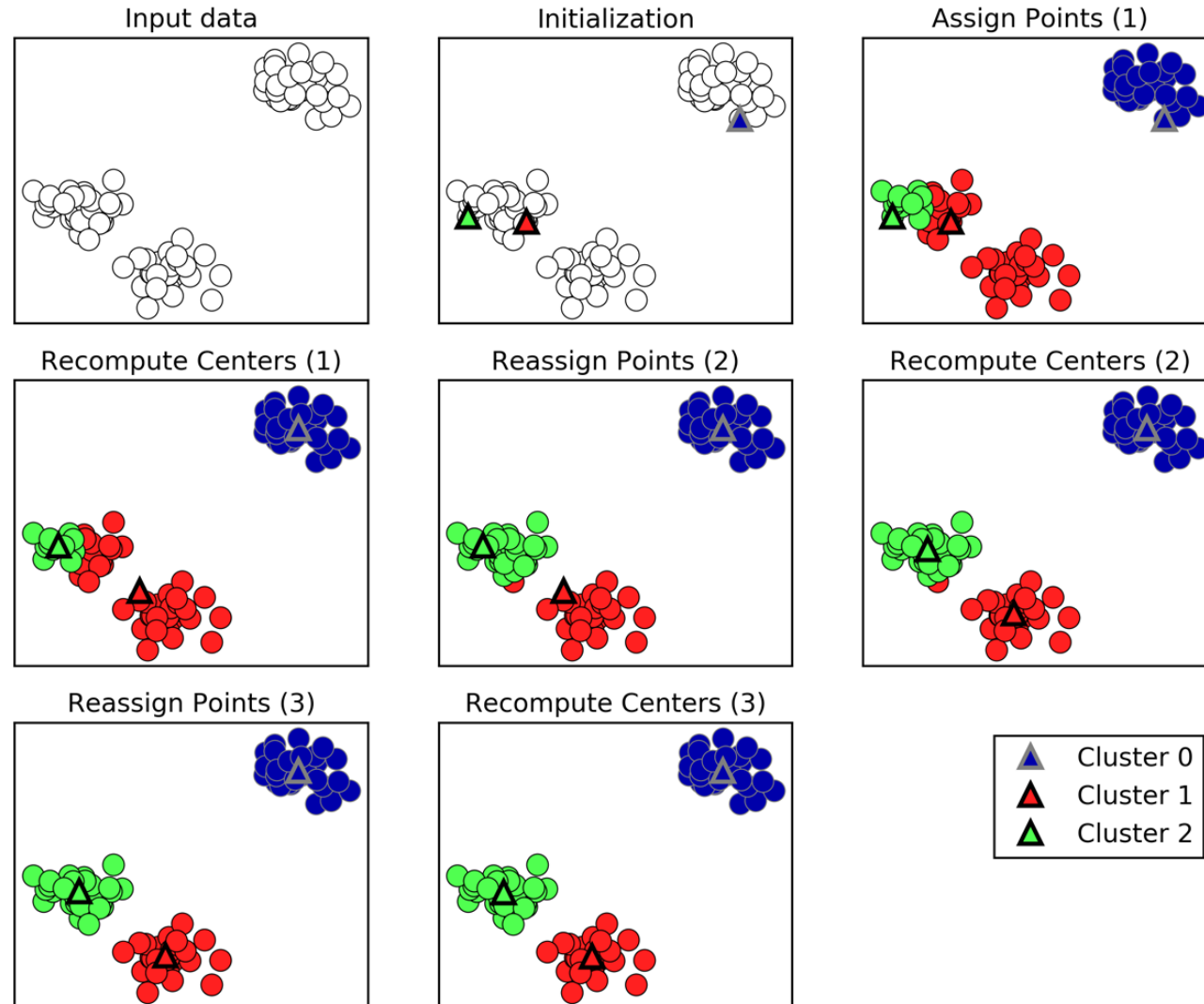
Then the iterative algorithm starts. First, each data point is assigned to the cluster center it is closest to (“Assign Points”).

k-Means Clustering



Next, the cluster centers are updated to be the mean of the assigned points (“Recompute Centers”).

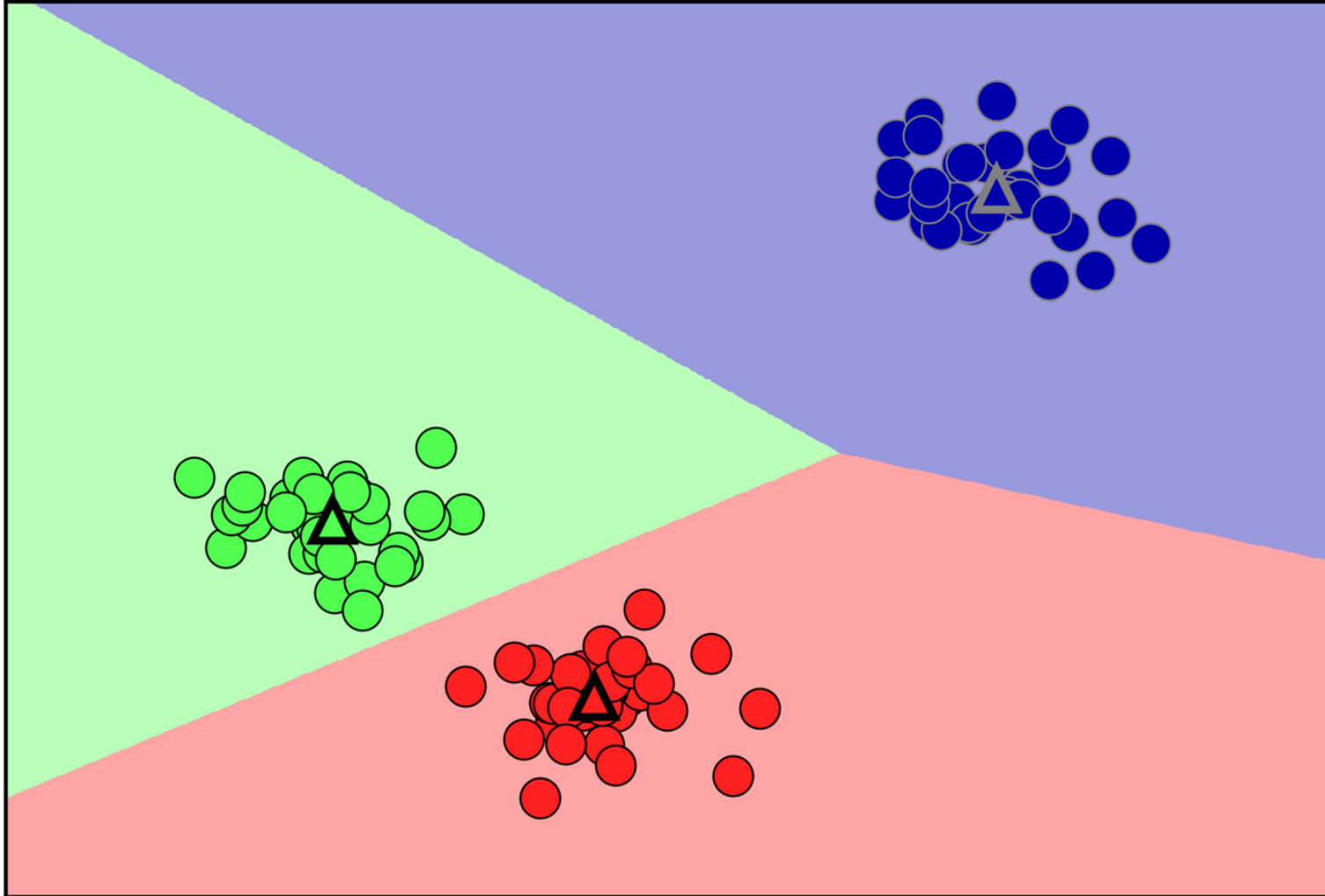
k-Means Clustering



Then the process is repeated two more times. After the third iteration, the assignment of points to cluster centers remained unchanged, so the algorithm stops.

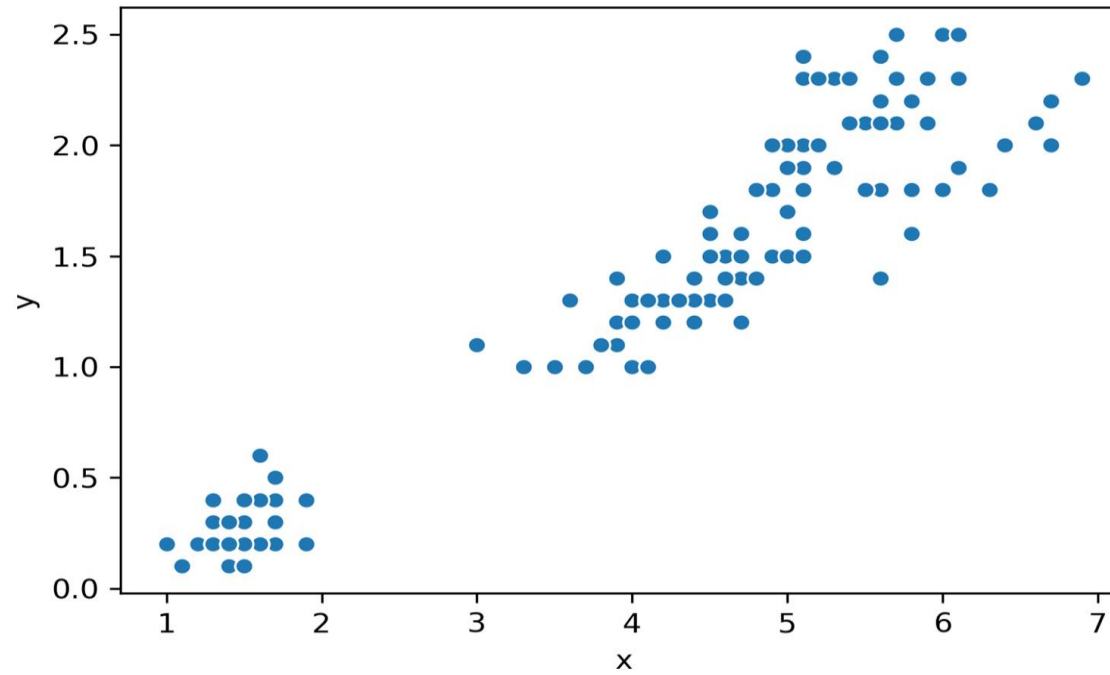
Given new data points, k -means will assign each to the closest cluster center.

k-Means Clustering

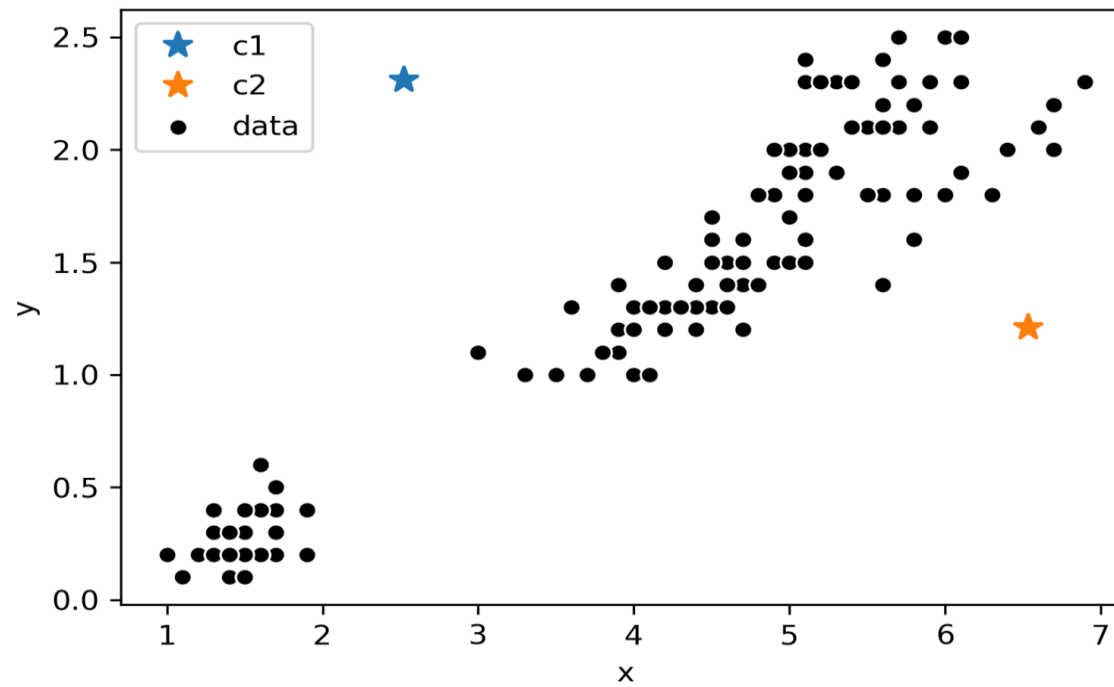


Cluster centers and cluster boundaries found by the k-means algorithm.

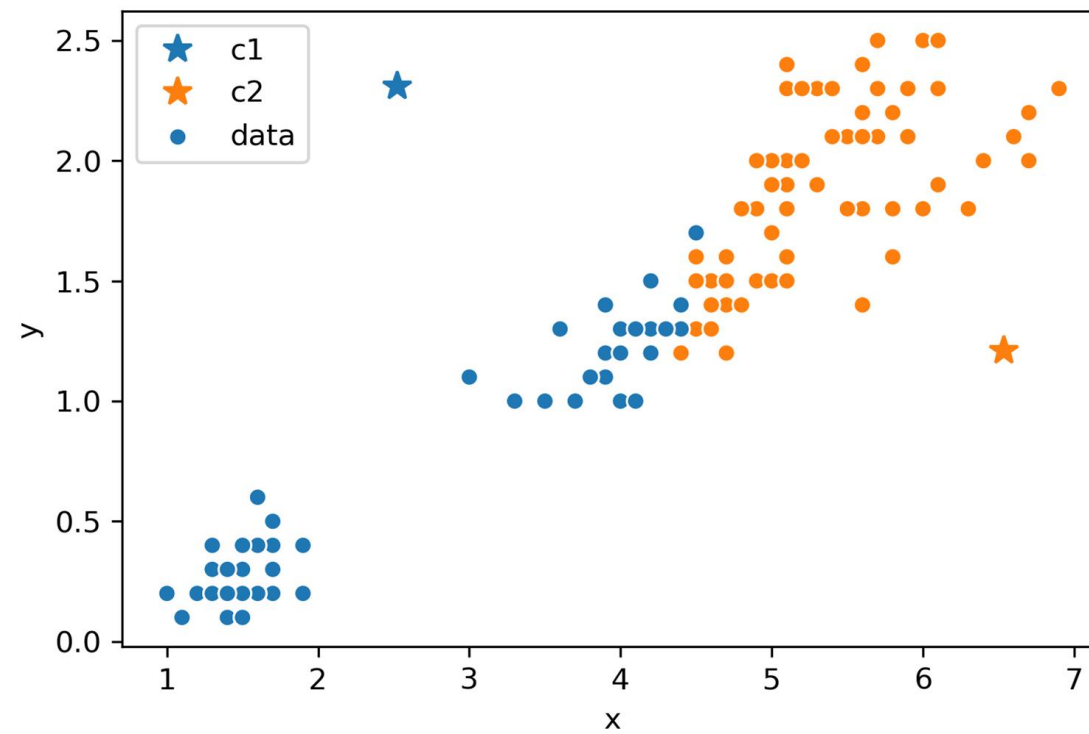
Raw data →



Initial random
placement of two
centers

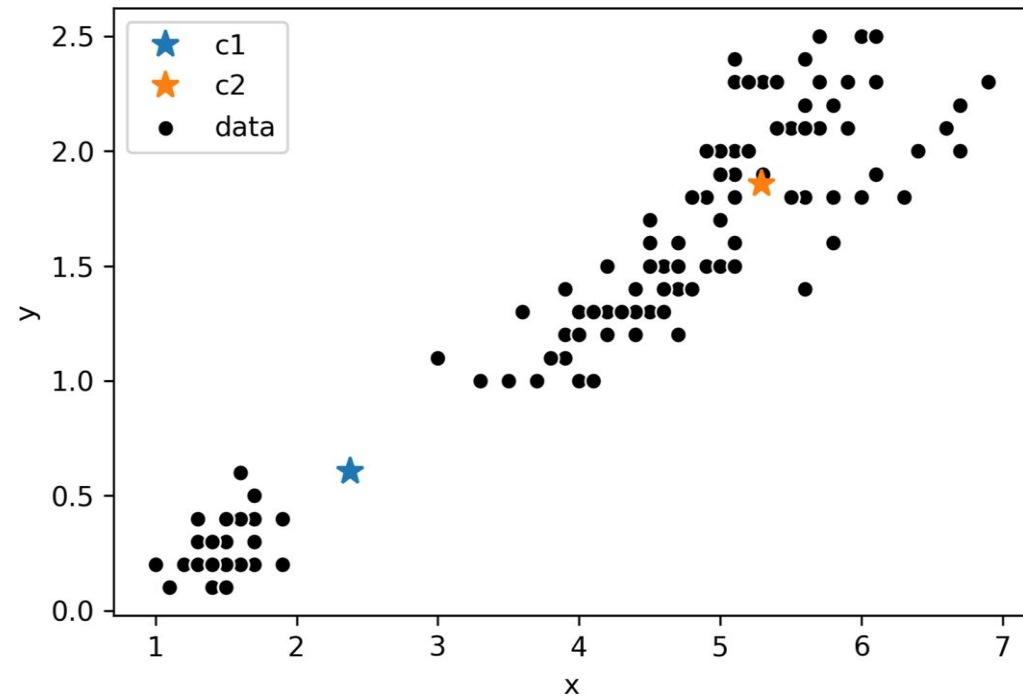


Data colored by
closest center



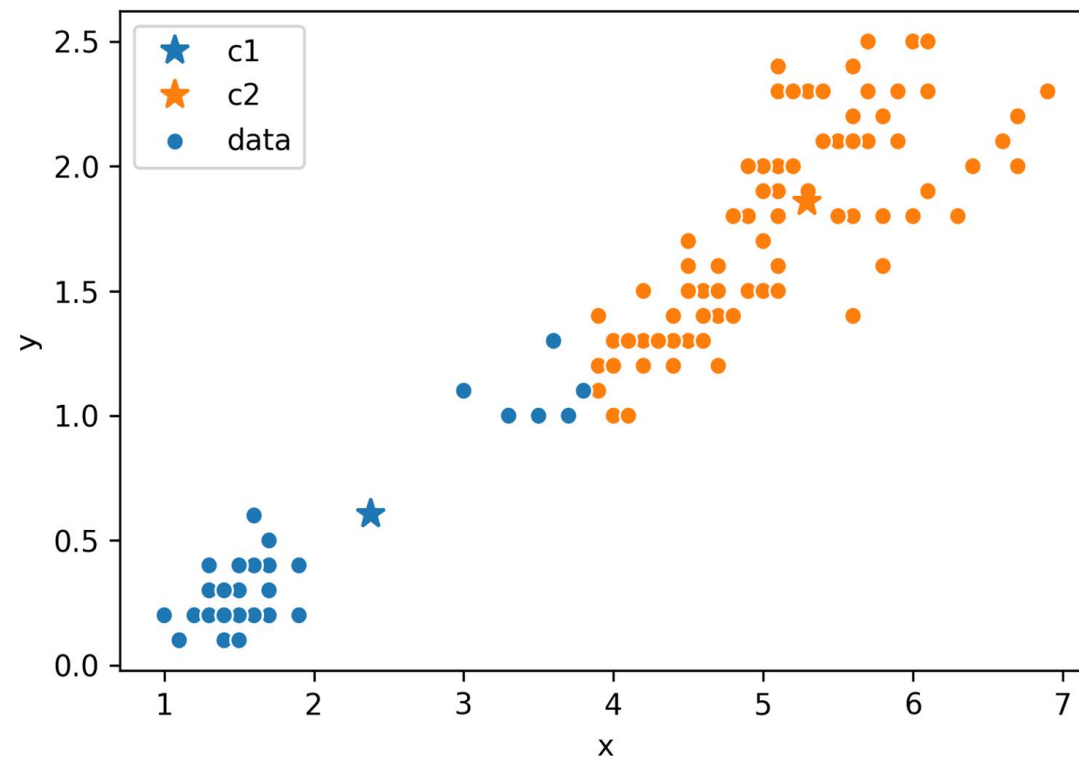
Iteration: 0

Centers moved to
new position



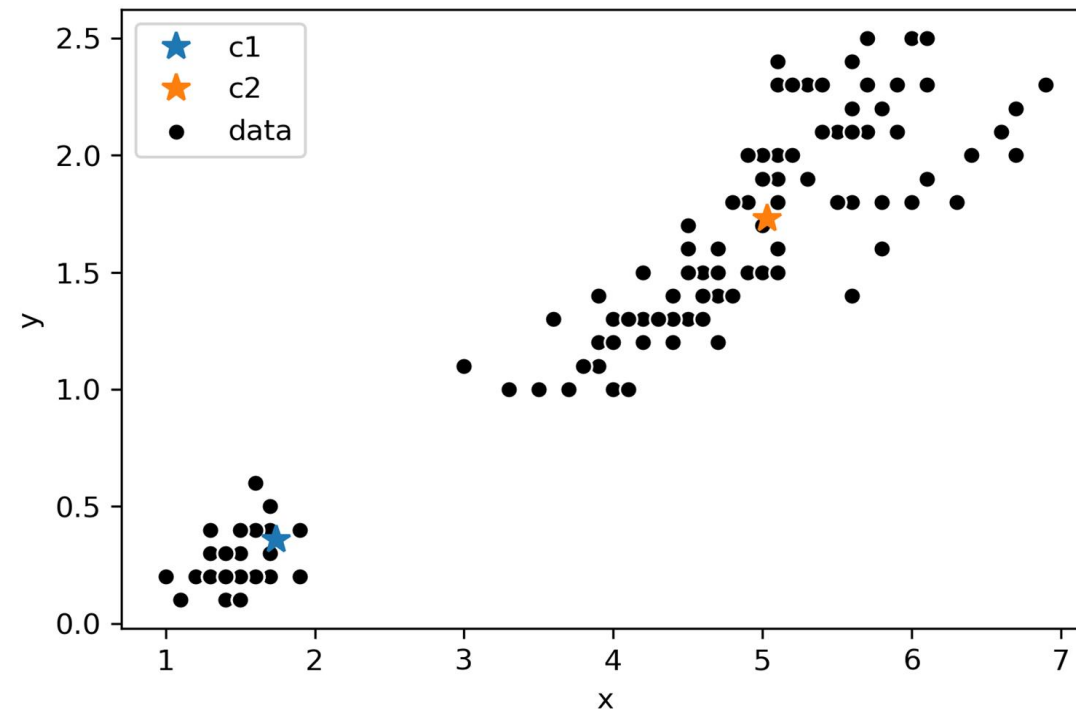
Iteration: 1

Data colored by
closest center (in
new position)



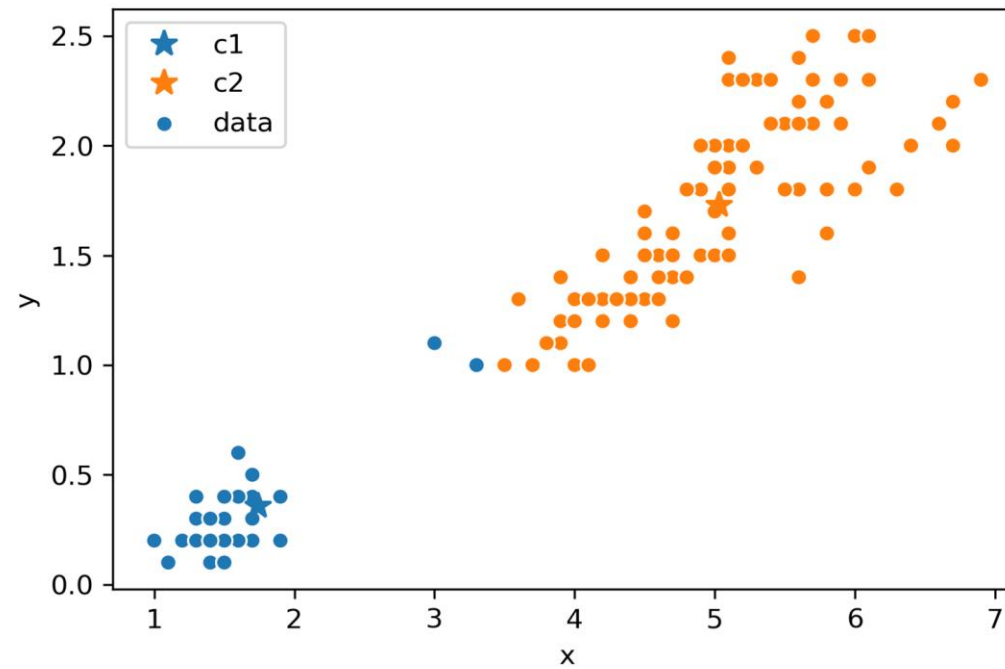
Iteration: 1

Centers moved to
new position



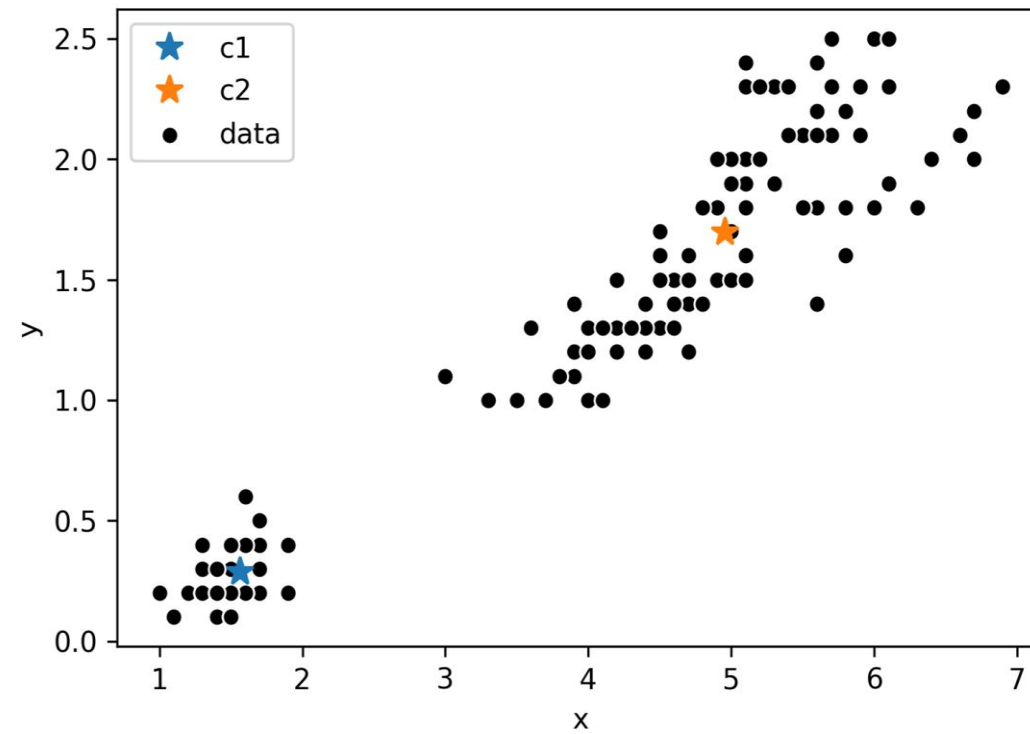
Iteration: 2

Data colored by
closest center (in
new position)



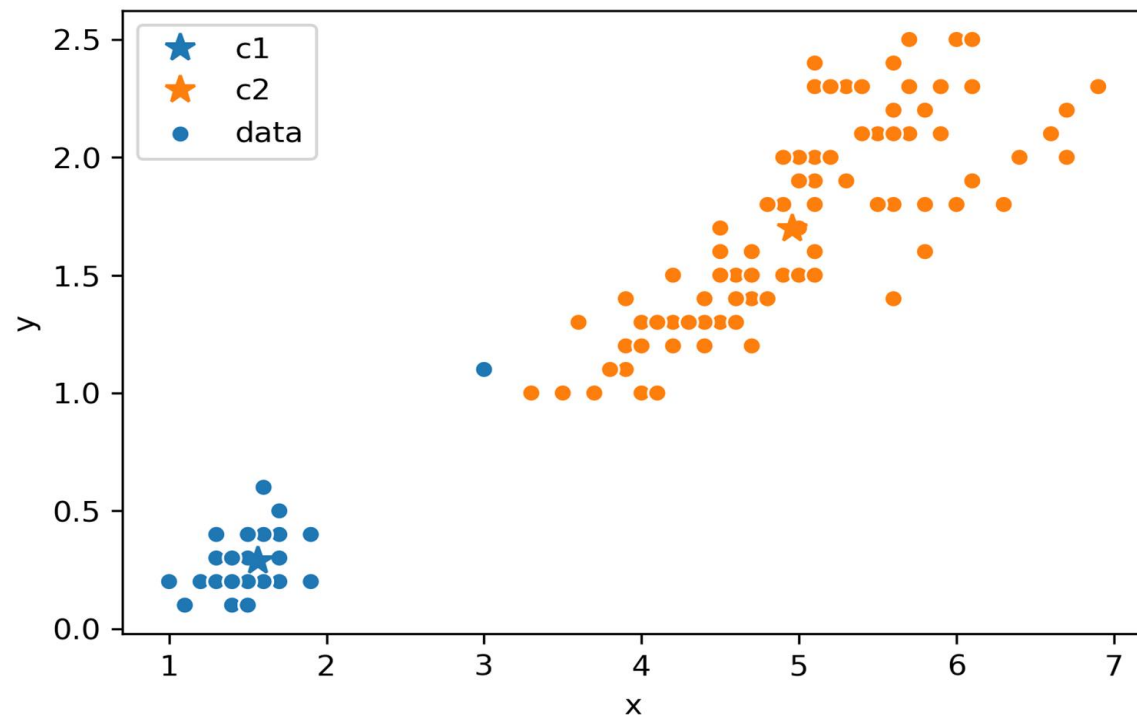
Iteration: 3

Centers moved to
new position



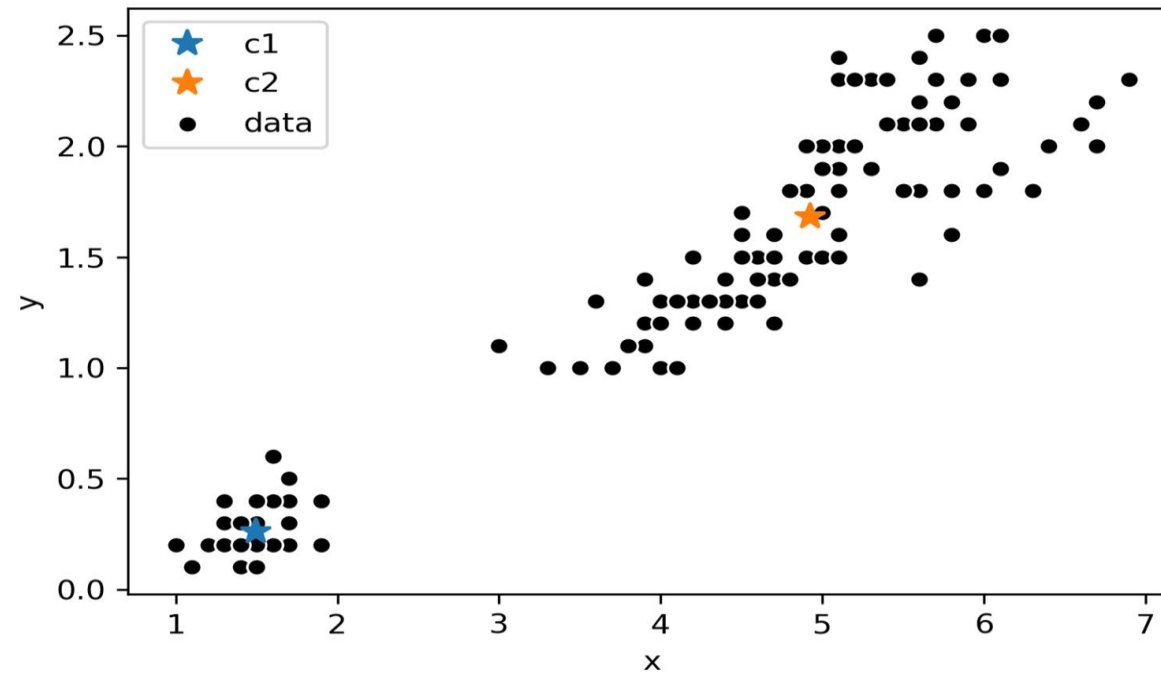
Iteration: 4

Data colored by
closest center (in
new position)



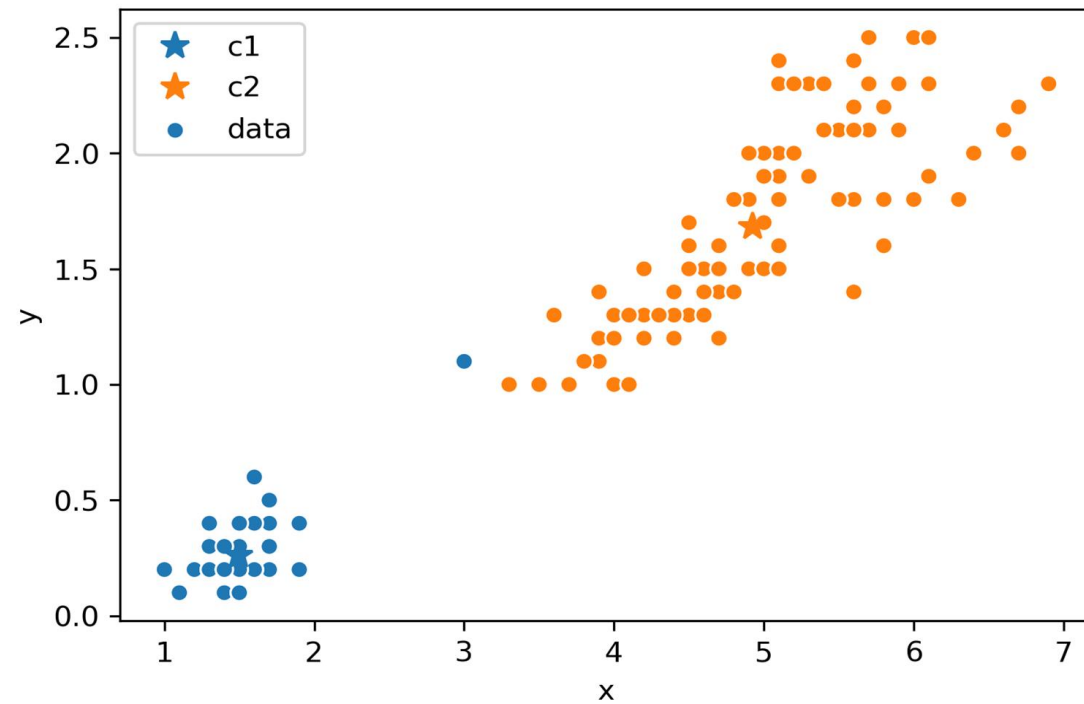
Iteration: 4

Centers moved to
new position

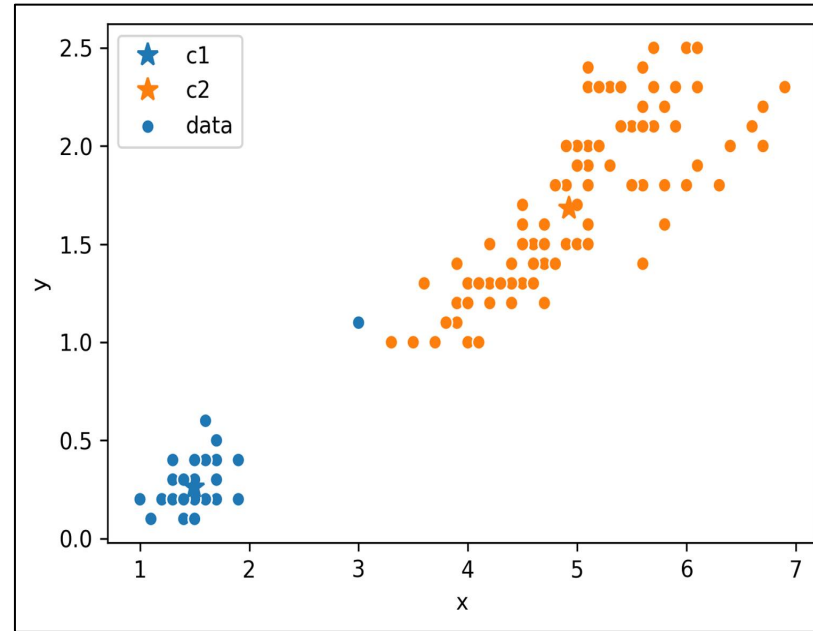
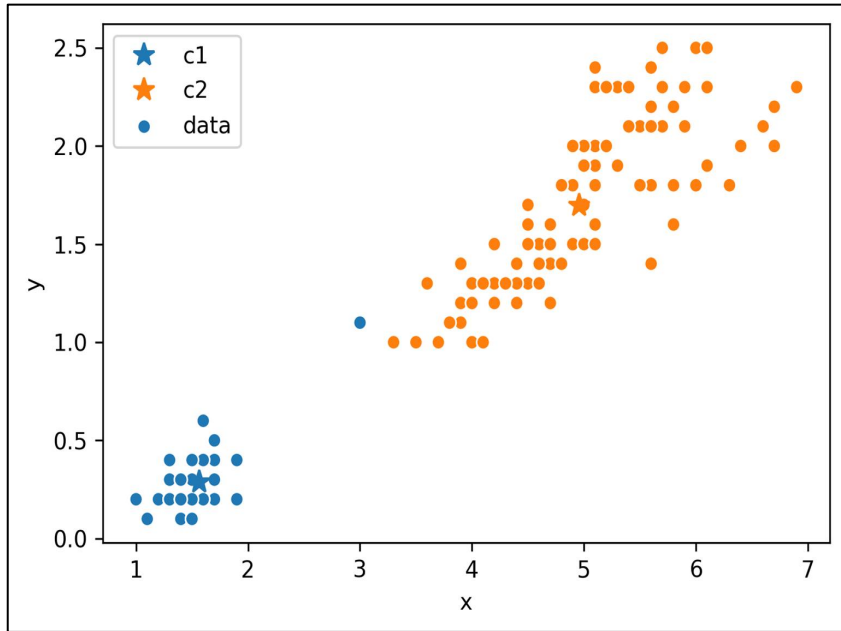


Iteration: 5

Data colored by
closest center (in
new position)



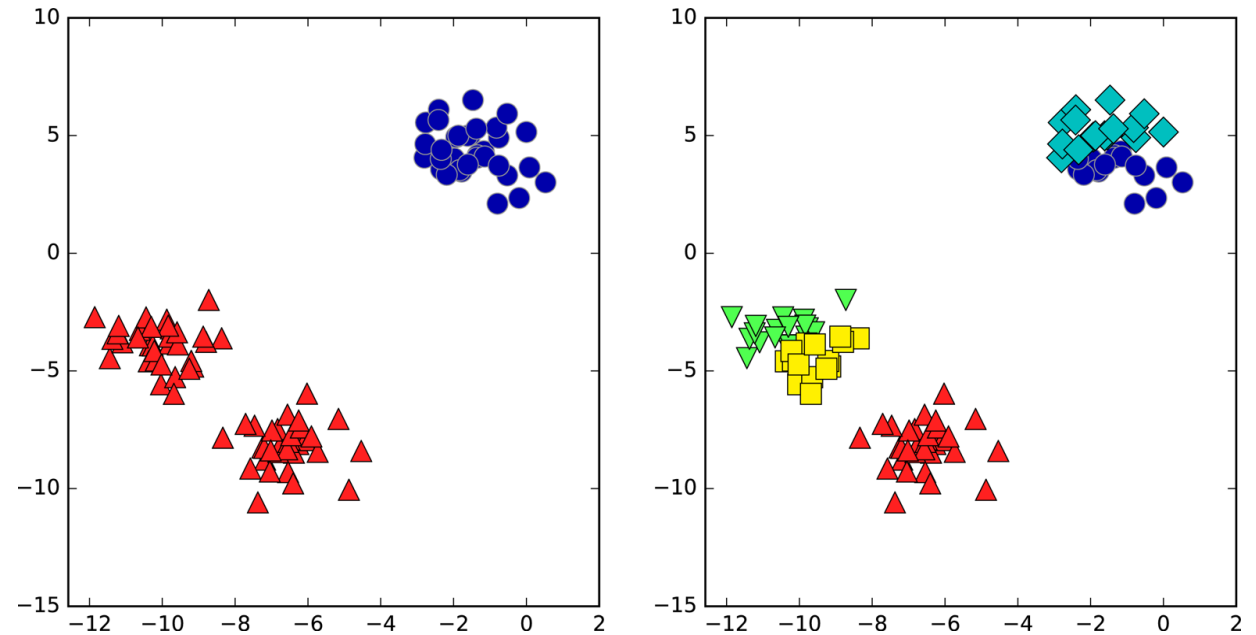
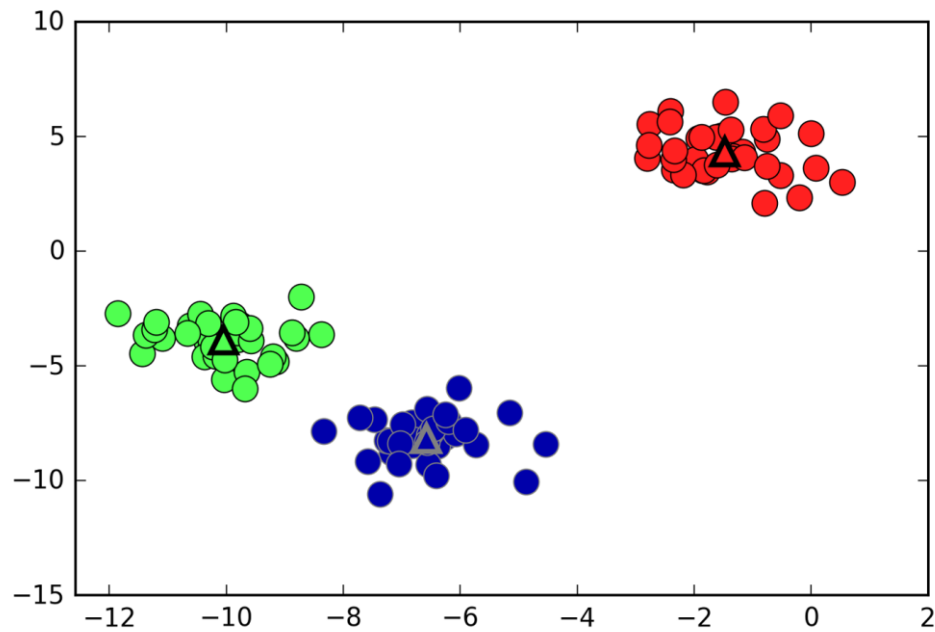
Iteration: 5



Above, we see the results after iteration 4 and 5:

- Centers moved slightly between iteration 4 and 5.
- But no points changed color.
- Are we done?
 - Yes! If we tried iteration 6, we'd see that centers don't move at all.

Picking number of cluster centers

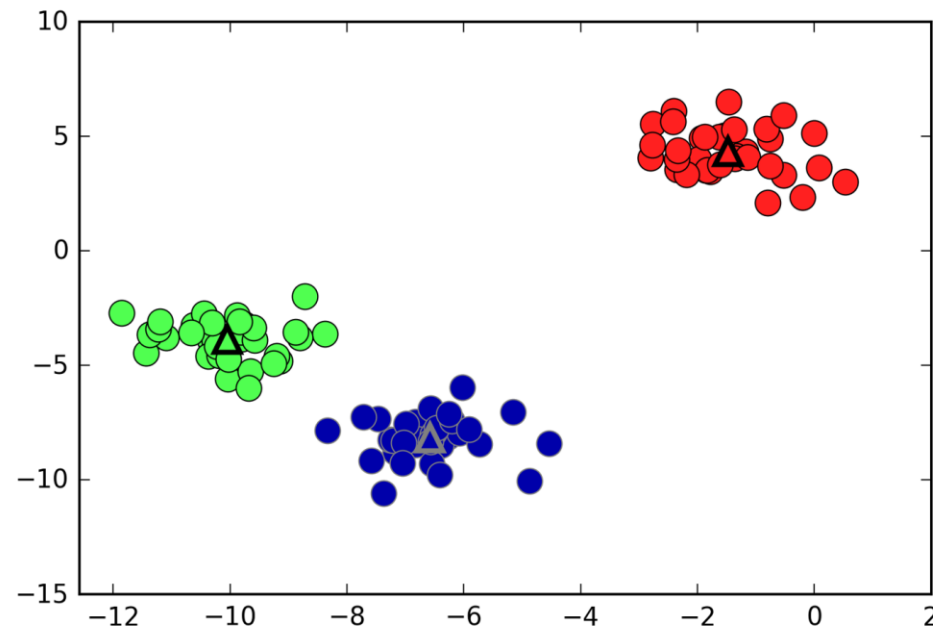


Cluster assignments found by k-means using two clusters (left) and five clusters (right)

Cluster labels themselves have no a priori meaning

There is **no ground truth**, and consequently, the labels themselves have no *a priori* meaning.

No significance to the fact that one group was labeled 0 and another one was labeled 1. Running the algorithm again might result in a different numbering of clusters because of the random nature of the initialization.



Cluster assignments and cluster centers found by k-means with three clusters

k-Means Clustering (Python Example)

We instantiate the k-Means class, and set the number of clusters we are looking for. Then we call the fit method with the data:

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
# generate synthetic two-dimensional data
X, y = make_blobs(random_state=1)
# build the clustering model
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
```

k-Means Clustering

During the algorithm, each training data point in X is assigned a cluster label. You can find these labels in the `kmeans.labels_` attribute:

```
print("Cluster memberships:\n{}".format(kmeans.labels_))
```

Cluster memberships:

```
[1 2 2 2 0 0 0 2 1 1 2 2 0 1 0 0 0 1 2 2 0 2 0 1 2 0 0 1 1 0 1 1 0 1 2 0 2  
2 2 0 0 2 1 2 2 0 1 1 1 1 2 0 0 0 1 0 2 2 1 1 2 0 0 2 2 0 1 0 1 2 2 2 0 1  
1 2 0 0 1 2 1 2 2 0 1 1 1 1 2 1 0 1 1 2 2 0 0 1 0 1]
```

As we asked for three clusters, the clusters are numbered 0 to 2.

k-Means Clustering

You can also assign cluster labels to new points, using the predict method. Each new point is assigned to the closest cluster center when predicting, but the existing model is not changed.

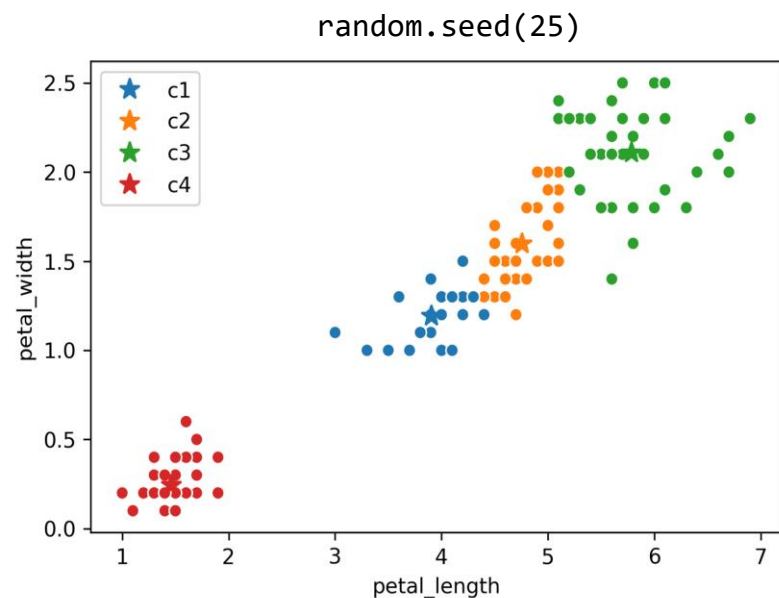
Running predict on the training set returns the same result as labels_:

```
print(kmeans.predict(X))
```

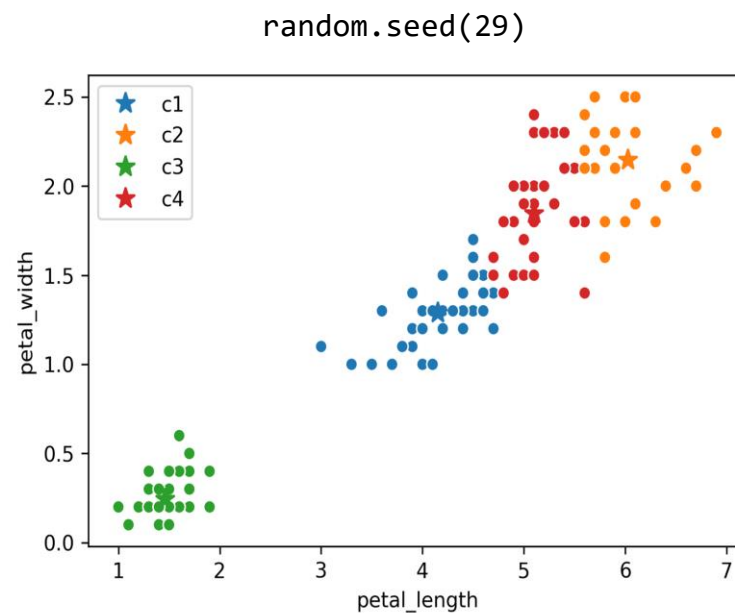
```
[1 2 2 2 0 0 0 2 1 1 2 2 0 1 0 0 0 1 2 2 0 2 0 1 2 0 0 1 1 0 1 1 0 1 2 0 2
 2 2 0 0 2 1 2 2 0 1 1 1 1 2 0 0 0 1 0 2 2 1 1 2 0 0 2 2 0 1 0 1 2 2 2 0 1
 1 2 0 0 1 2 1 2 2 0 1 1 1 1 2 1 0 1 1 2 2 0 0 1 0 1]
```

You can see that clustering is somewhat similar to classification, in that each item gets a **label**. However, there is **no ground truth**, and consequently the labels themselves have no *a priori* meaning.

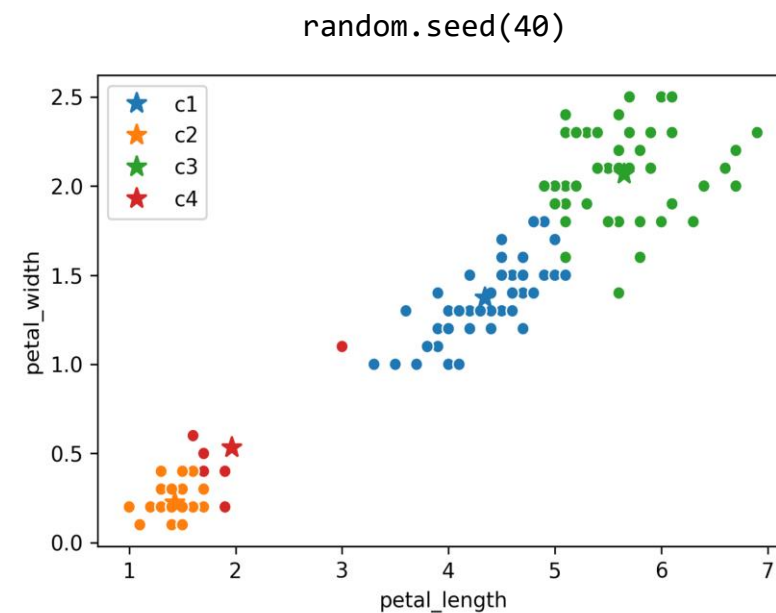
Each time you run K-Means, you get a different output, depending on where centers started.



(A)



(B)



(C)

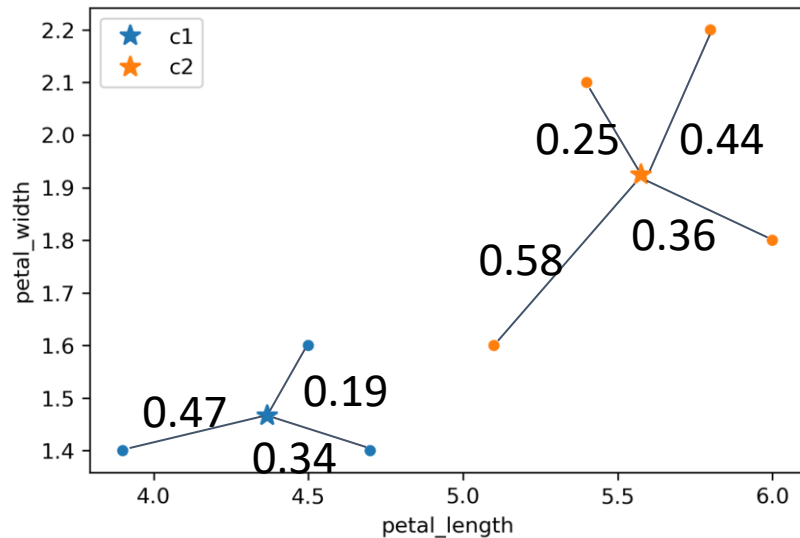
Which is best?

- One approach: Define some sort of loss function.

To evaluate different clustering results, we need a loss function.

Two common loss functions:

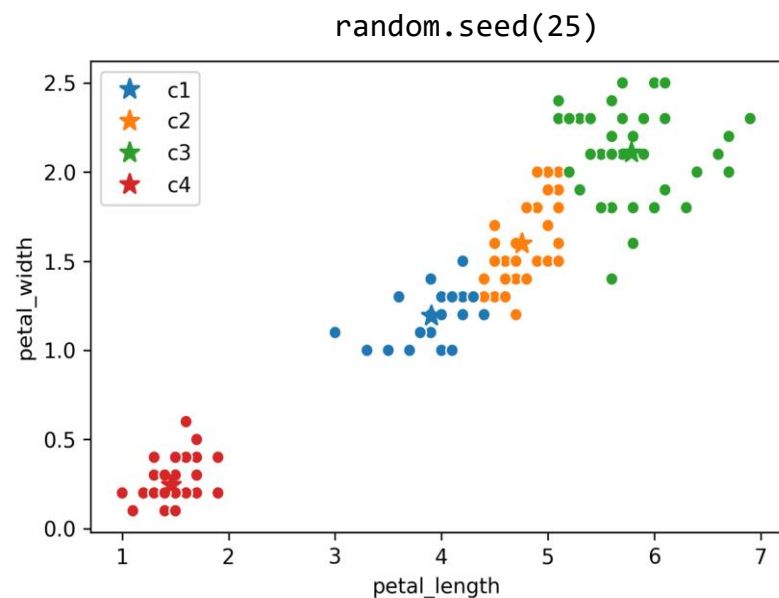
- **Inertia**: Sum of squared distances from each data point to its center.
- **Distortion**: Weighted sum of squared distances from each data point to its center.



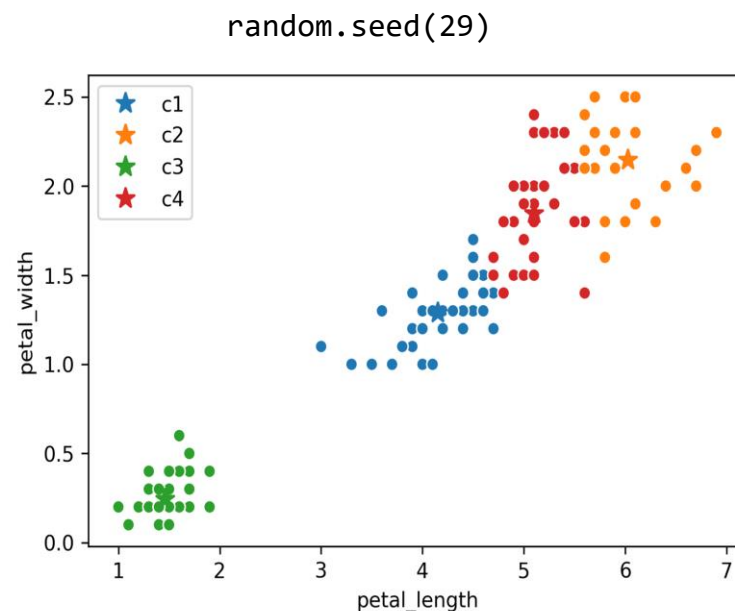
Example:

- **Inertia**: $0.47^2 + 0.19^2 + 0.34^2 + 0.25^2 + 0.58^2 + 0.36^2 + 0.44^2$
- **Distortion**: $(0.47^2 + 0.19^2 + 0.34^2)/3 + (0.25^2 + 0.58^2 + 0.36^2 + 0.44^2)/4$

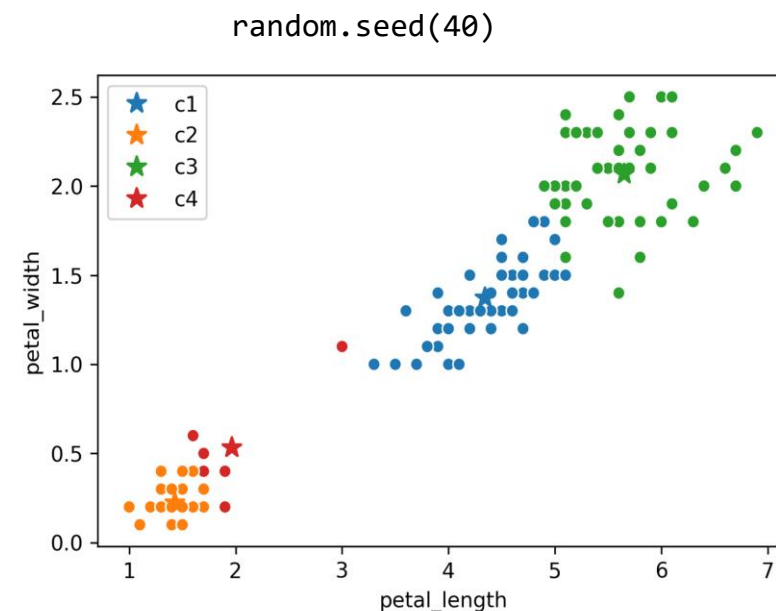
Each time you run K-Means, you get a different output, depending on where centers started.



Inertia:
44.96



Inertia:
45.95



Inertia:
54.35

Which is best?

Among these three choices, our **inertia** loss function says that the leftmost clustering is best (inertia: 44.96) and rightmost clustering (inertia: 54.35) is worst.

k-Means Clustering for Image Segmentation

$K = 2$



$K = 3$



$K = 10$



Original image



Failure cases of k-means

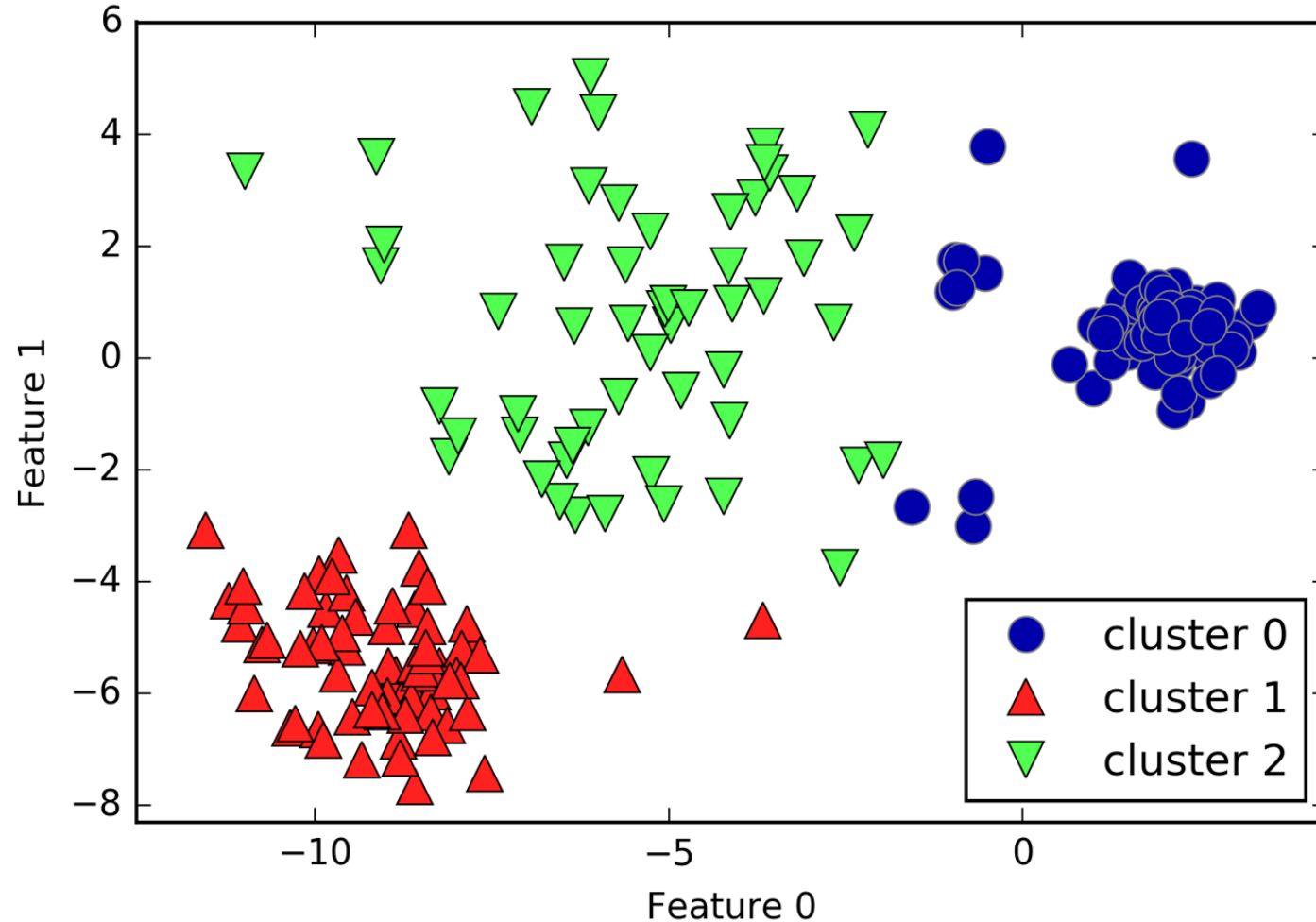
Even if you know the “right” number of clusters for a given dataset, k-means might not always be able to recover them.

Each cluster is defined solely by its center, which means that each cluster is a convex shape. As a result of this, k-means can only capture relatively simple shapes.

k-means also assumes that all clusters have the same “diameter” in some sense; it always draws the boundary between clusters to be exactly in the middle between the cluster centers.

-> does not always work

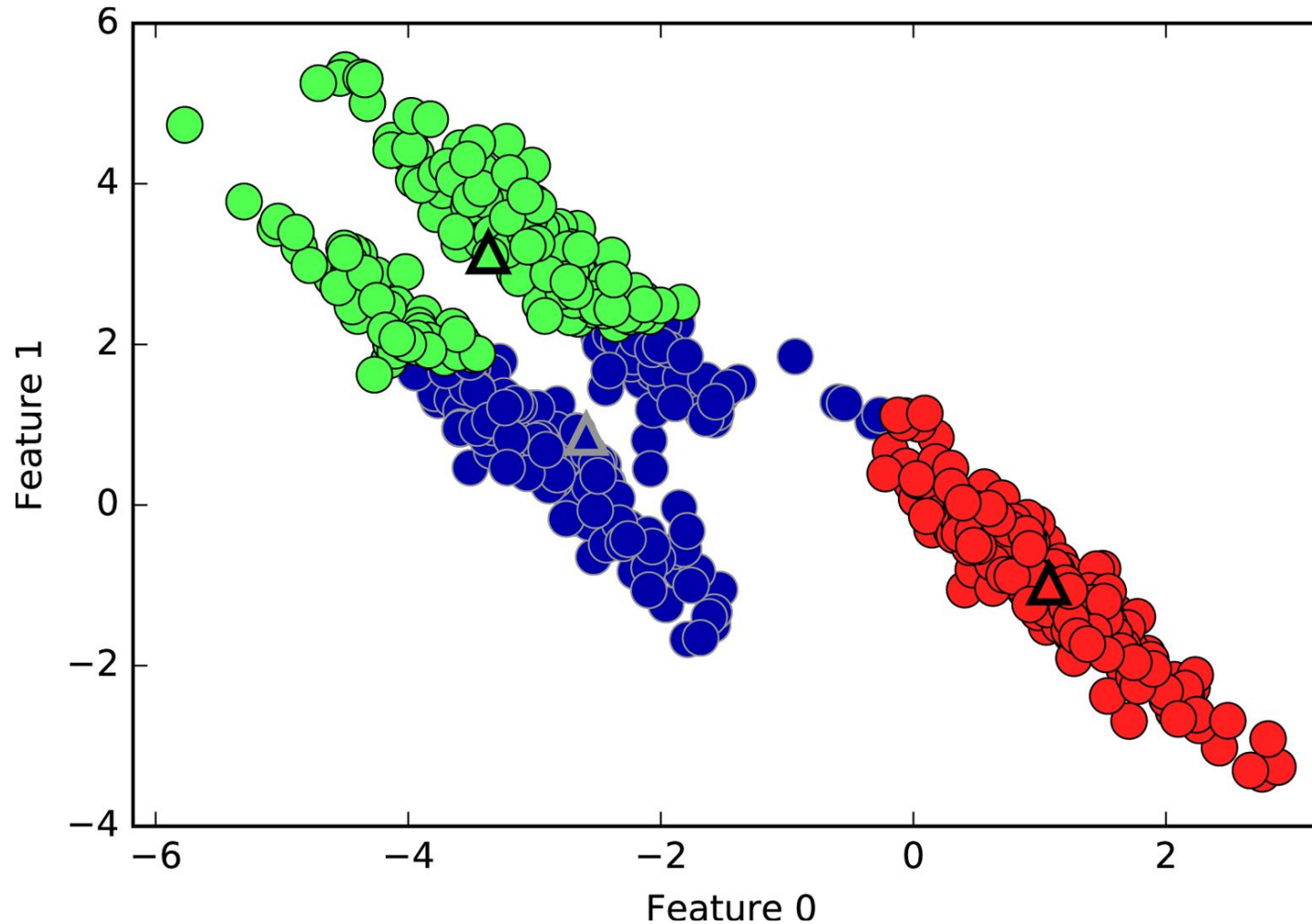
Failure cases of k-means



Both cluster 0 and cluster 1 have some points that are far away from all the other points in these clusters that “reach” toward the center.

Cluster assignments found by k-means when clusters have different densities.

Failure cases of k-means

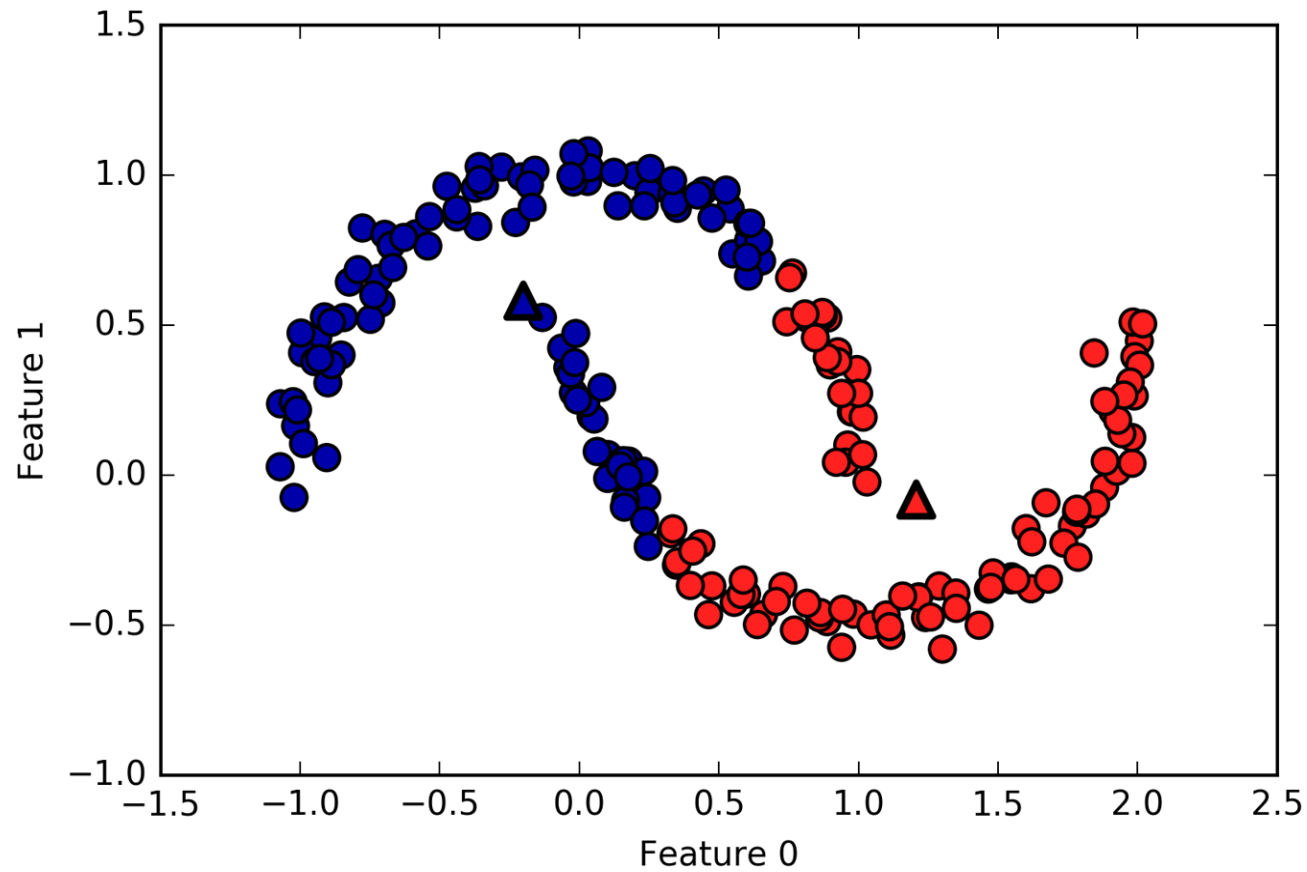


k-means also assumes that all directions are equally important for each cluster.

The groups shown are stretched toward the diagonal. As k-means only considers the distance to the nearest cluster center, it can't handle this kind of data

k-means fails to identify nonspherical clusters

Failure cases of k-means



k-means also assumes that all directions are equally important for each cluster.

The groups shown are stretched toward the diagonal. As k-means only considers the distance to the nearest cluster center, it can't handle this kind of data

k-means also performs poorly if the clusters have more complex shapes

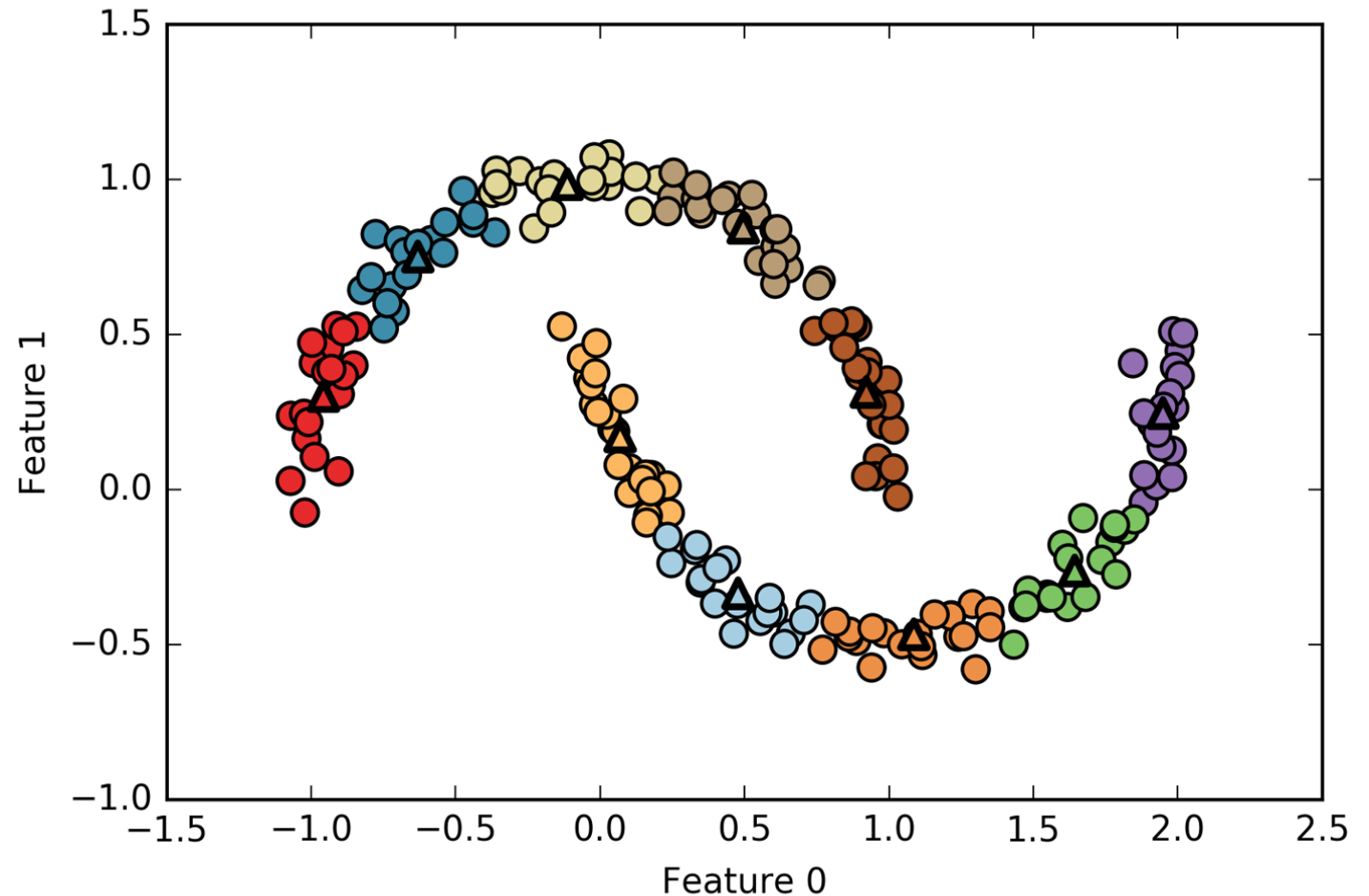
Vector quantization, or seeing k-means as decomposition

k-means tries to represent each data point using a cluster center.

- You can think of that as each point being represented using only a single component, which is given by the cluster center. This view of k-means as a decomposition method, where each point is represented using a single component, is called vector quantization.
- An interesting aspect of vector quantization using k-means is that we can use many more clusters than input dimensions to encode our data.

Vector quantization, or seeing k-means as decomposition

- An interesting aspect of vector quantization using k-means is that we can use many more clusters than input dimensions to encode our data.



Using many k-means clusters to cover the variation in a complex dataset

K-Means advantages and disadvantages

k-means is a very popular algorithm for clustering, not only because it is relatively easy to understand and implement, but also because it runs relatively quickly. K-means scales easily to large datasets.

One of the drawbacks of k-means is that it relies on a random initialization, which means the outcome of the algorithm depends on a random seed.

Further downsides of k-means are the relatively restrictive assumptions made on the shape of clusters, and the requirement to specify the number of clusters you are looking for (which might not be known in a real-world application).

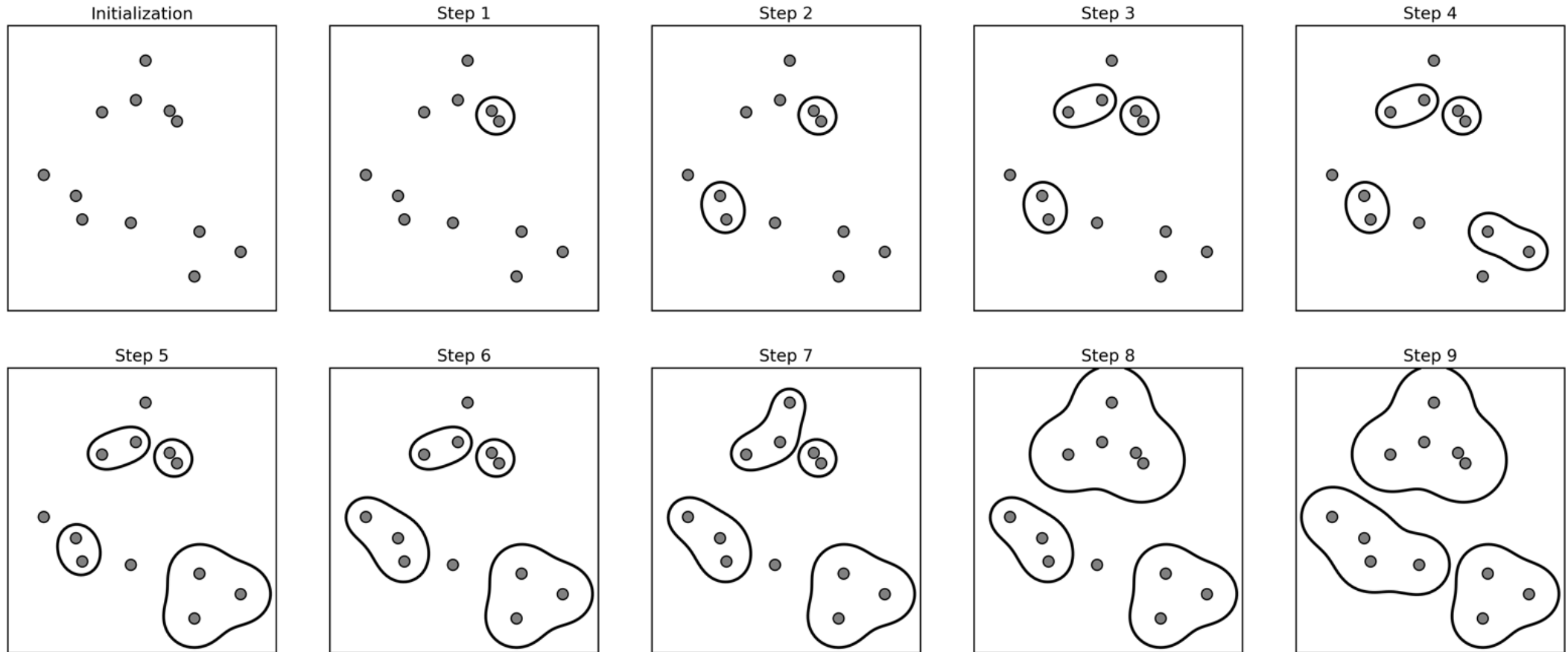
Agglomerative Clustering

Agglomerative clustering refers to a collection of clustering algorithms that all build upon the same principles: the algorithm starts by declaring each point its own cluster, and then merges the two most similar clusters until some stopping criterion is satisfied.

- e.g. the number of clusters: similar clusters are merged until only the specified number of clusters are left.

There are several linkage criteria that specify how exactly the “most similar cluster” is measured. This measure is always defined between two existing clusters.

Agglomerative Clustering

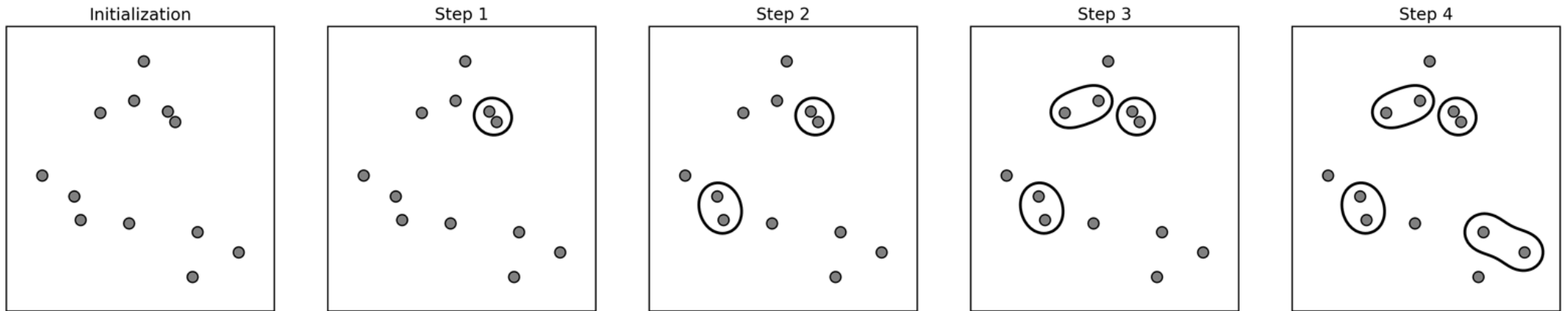


Agglomerative clustering iteratively joins the two closest clusters

Agglomerative Clustering

Initially, each point is its own cluster. Then, in each step, the two clusters that are closest are merged.

In the first four steps, two single-point clusters are picked and these are joined into two-point clusters

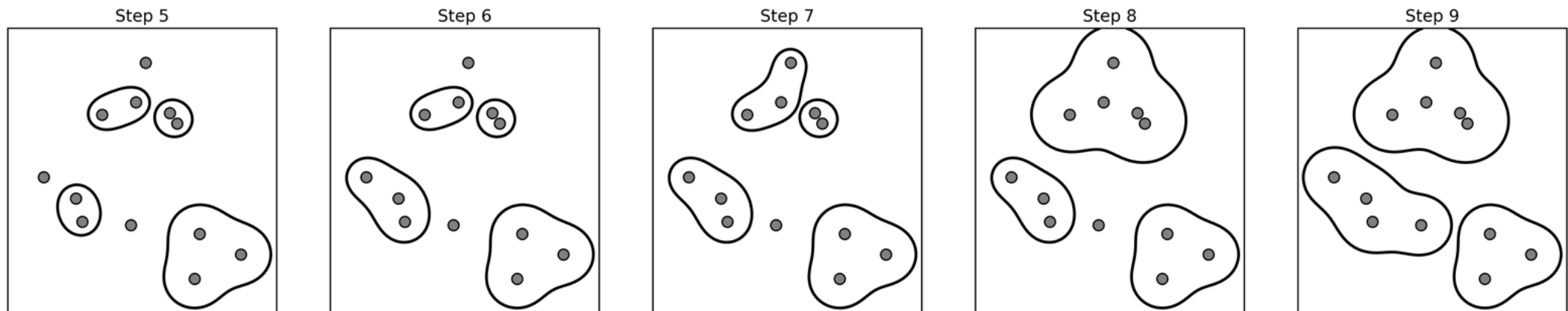


Agglomerative Clustering

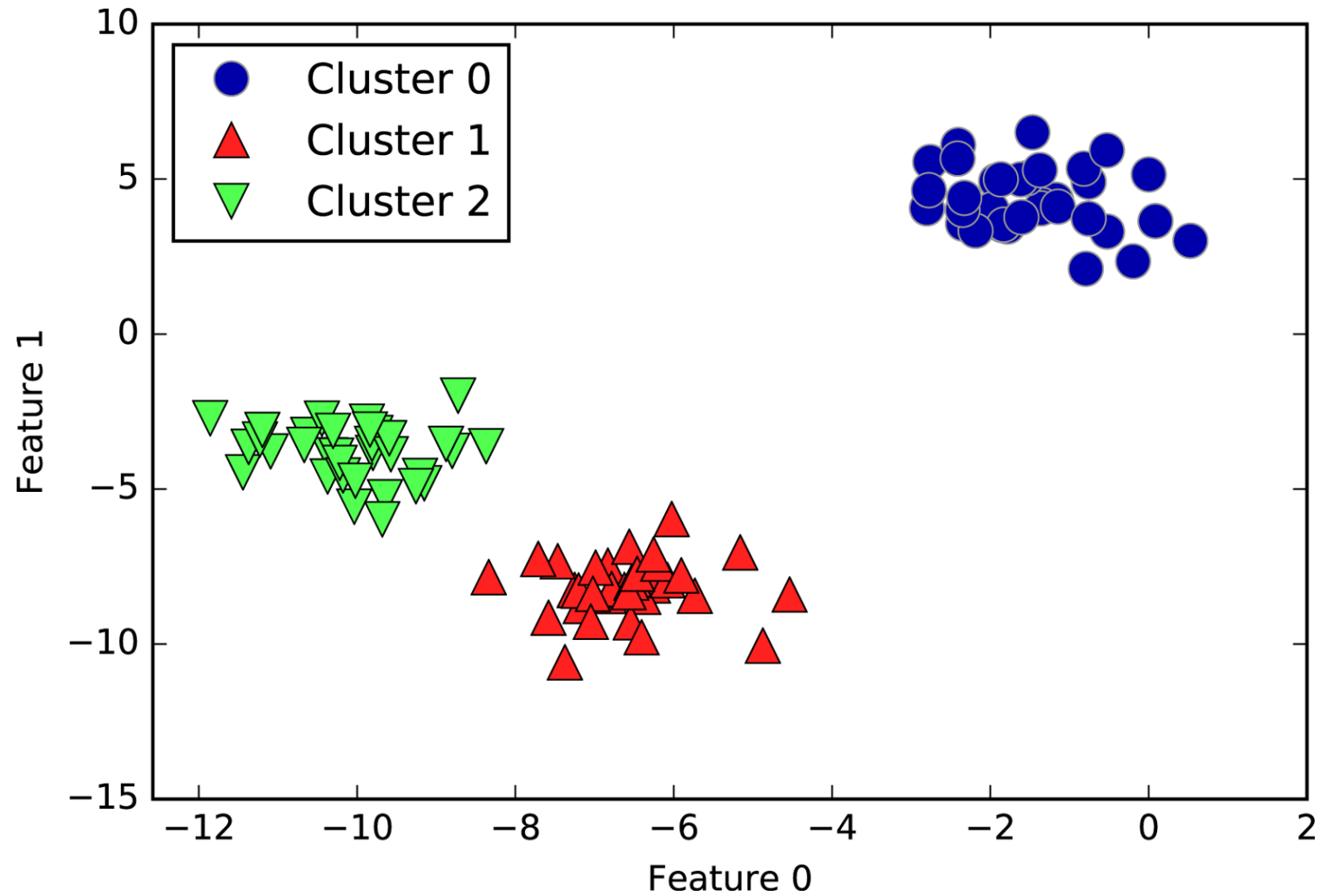
In step 5, one of the two-point clusters is extended to a third point, and so on. In step 9, there are only three clusters remaining.

As we specified that we are looking for three clusters, the algorithm then stops.

- Because of the way the algorithm works, agglomerative clustering cannot make predictions for new data points.



Agglomerative Clustering



Cluster assignment using agglomerative clustering with three clusters

Hierarchical clustering and dendrograms

Agglomerative clustering produces what is known as a hierarchical clustering.

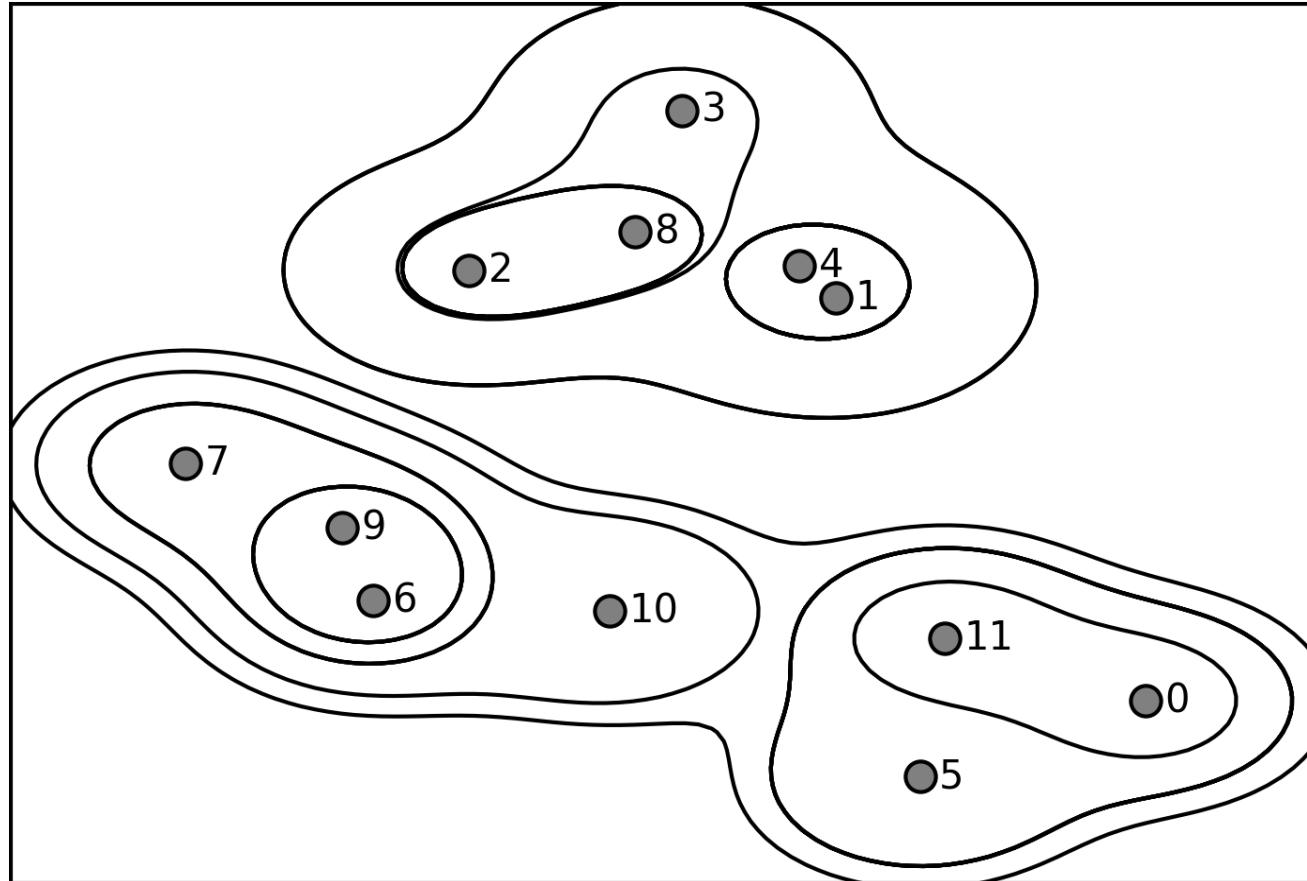
The clustering proceeds iteratively, and every point makes a journey from being a single point cluster to belonging to some final cluster.

Each intermediate step provides a clustering of the data (with a different number of clusters).

- It is sometimes helpful to look at all possible clusterings jointly.

Hierarchical clustering and dendrograms

An overlay of all the possible clusterings providing some insight into how each cluster breaks up into smaller clusters.



Hierarchical cluster assignment (shown as lines) generated with agglomerative clustering, with numbered data points

Hierarchical clustering and dendrograms

While this visualization provides a very detailed view of the hierarchical clustering, it relies on the two-dimensional nature of the data and therefore cannot be used on datasets that have more than two features.

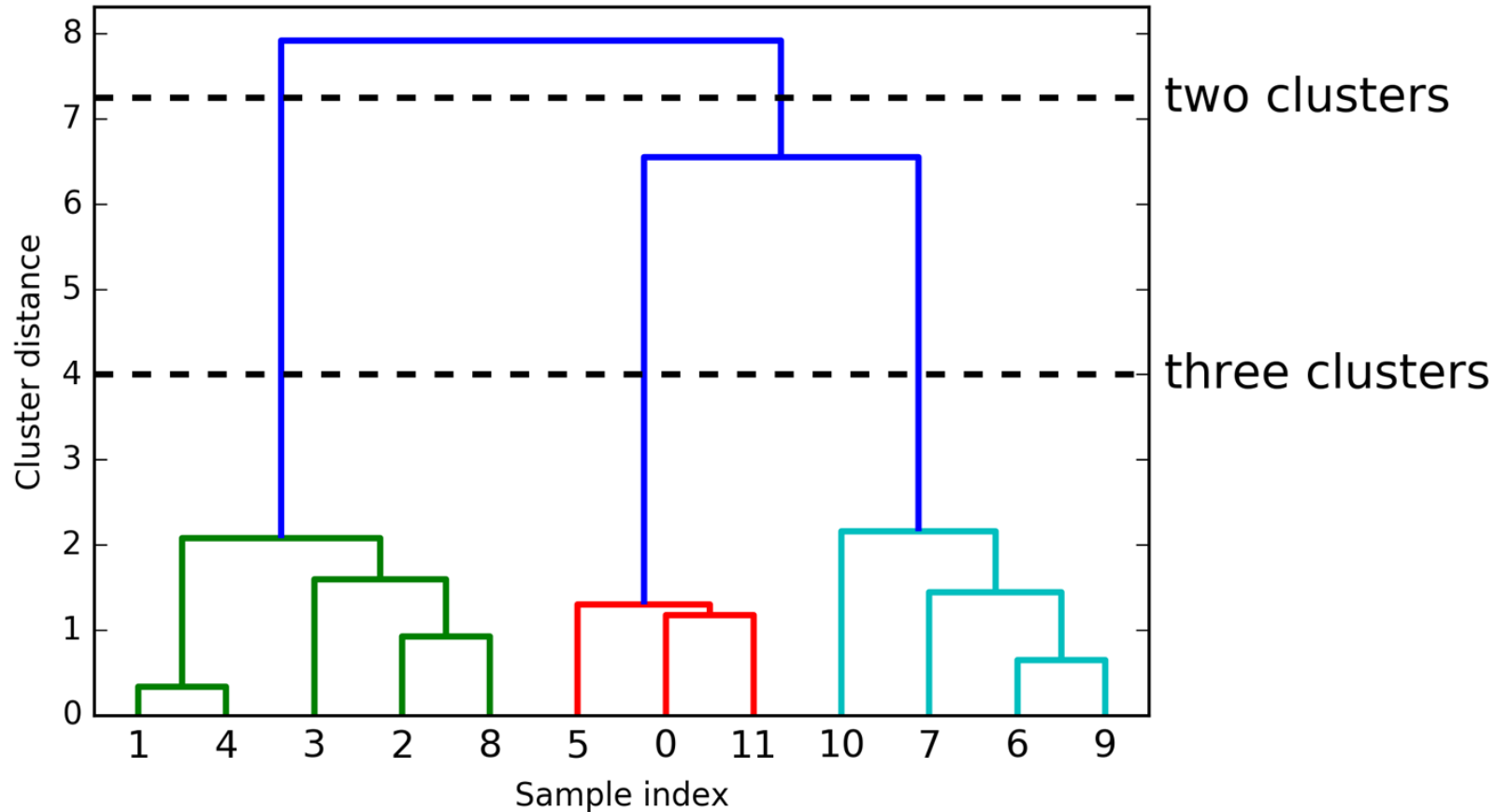
There is, however, another tool to visualize hierarchical clustering, called a dendrogram, that can handle multidimensional datasets.

SciPy provides a function that takes a data array X and computes a *linkage array*, which encodes hierarchical cluster similarities. We can then feed this linkage array into the `scipy dendrogram` function to plot the dendrogram.

Hierarchical clustering and dendrograms

```
# Import the dendrogram function and the ward clustering function from SciPy
from scipy.cluster.hierarchy import dendrogram, ward
X, y = make_blobs(random_state=0, n_samples=12)
# Apply the ward clustering to the data array X
# The SciPy ward function returns an array that specifies the distances
# bridged when performing agglomerative clustering
linkage_array = ward(X)
# Now we plot the dendrogram for the linkage_array containing the distances
# between clusters
dendrogram(linkage_array)
# Mark the cuts in the tree that signify two or three clusters
ax = plt.gca()
bounds = ax.get_xbound()
ax.plot(bounds, [7.25, 7.25], '--', c='k')
ax.plot(bounds, [4, 4], '--', c='k')
ax.text(bounds[1], 7.25, ' two clusters', va='center', fontdict={'size': 15})
ax.text(bounds[1], 4, ' three clusters', va='center', fontdict={'size': 15})
plt.xlabel("Sample index")
plt.ylabel("Cluster distance")
```

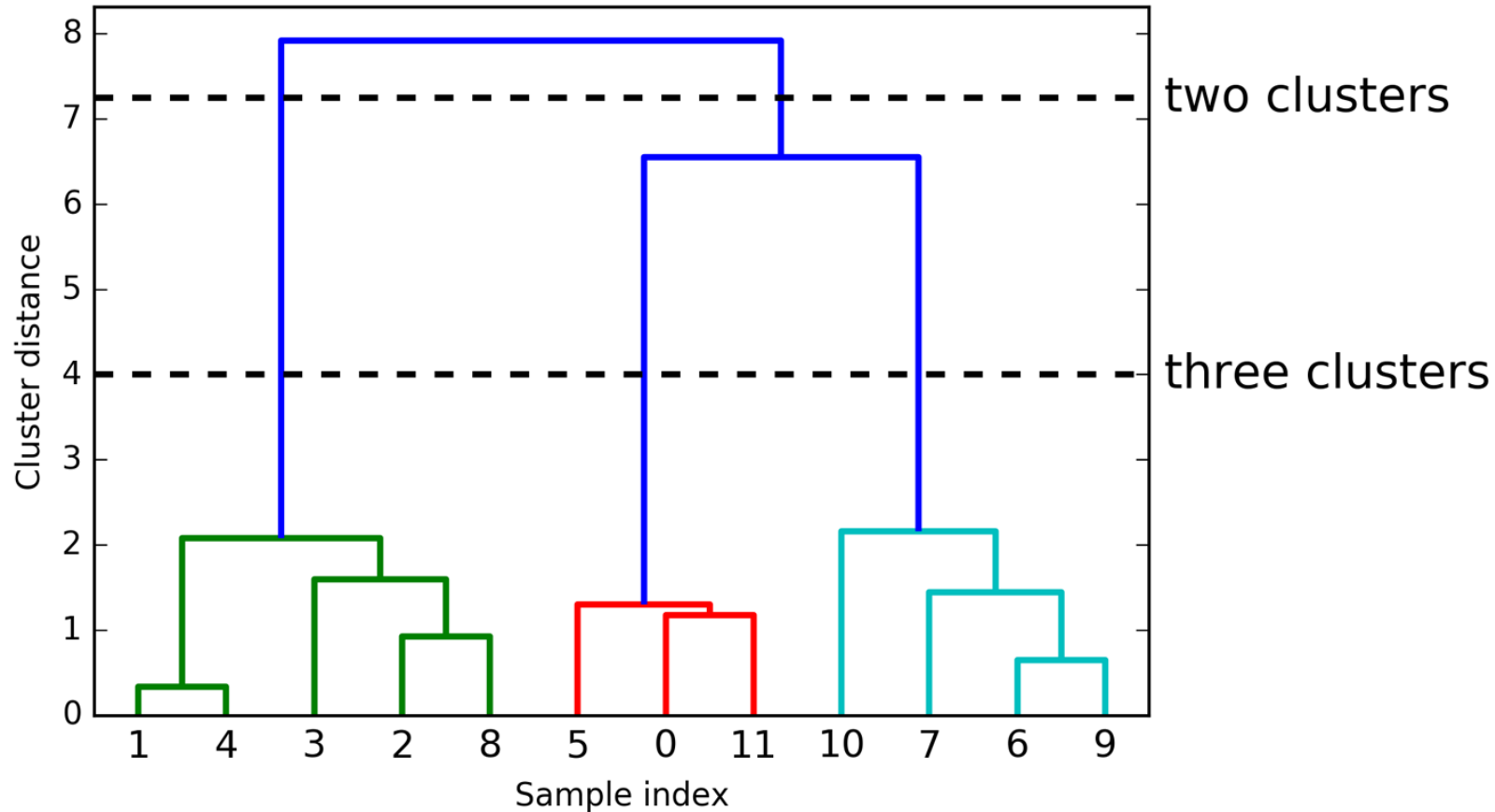
Hierarchical clustering and dendrograms



The dendrogram shows data points as points on the bottom (numbered from 0 to 11). Then, a tree is plotted with these points (representing single-point clusters) as the leaves, and a new node parent is added for each two clusters that are joined.

Dendrogram of the clustering shown with lines indicating splits into two and three clusters

Hierarchical clustering and dendrograms

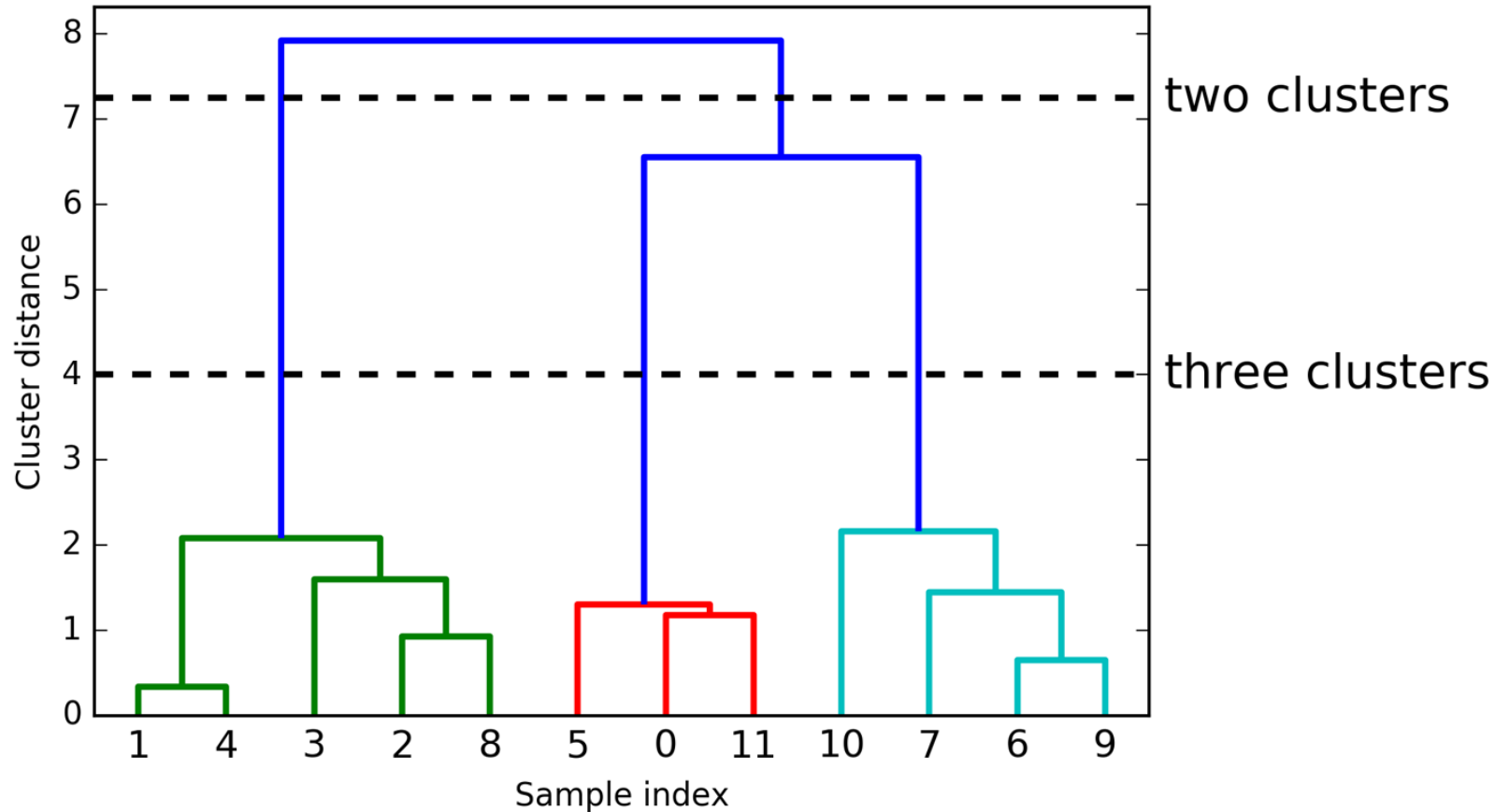


Reading from bottom to top, the data points 1 and 4 are joined first.

Next, points 6 and 9 are joined into a cluster, and so on. At the top level, there are two branches, one consisting of points 11, 0, 5, 10, 7, 6, and 9, and the other consisting of points 1, 4, 3, 2, and 8. These correspond to the two largest clusters in the lefthand side of the plot.

Dendrogram of the clustering shown with lines indicating splits into two and three clusters

Hierarchical clustering and dendrograms

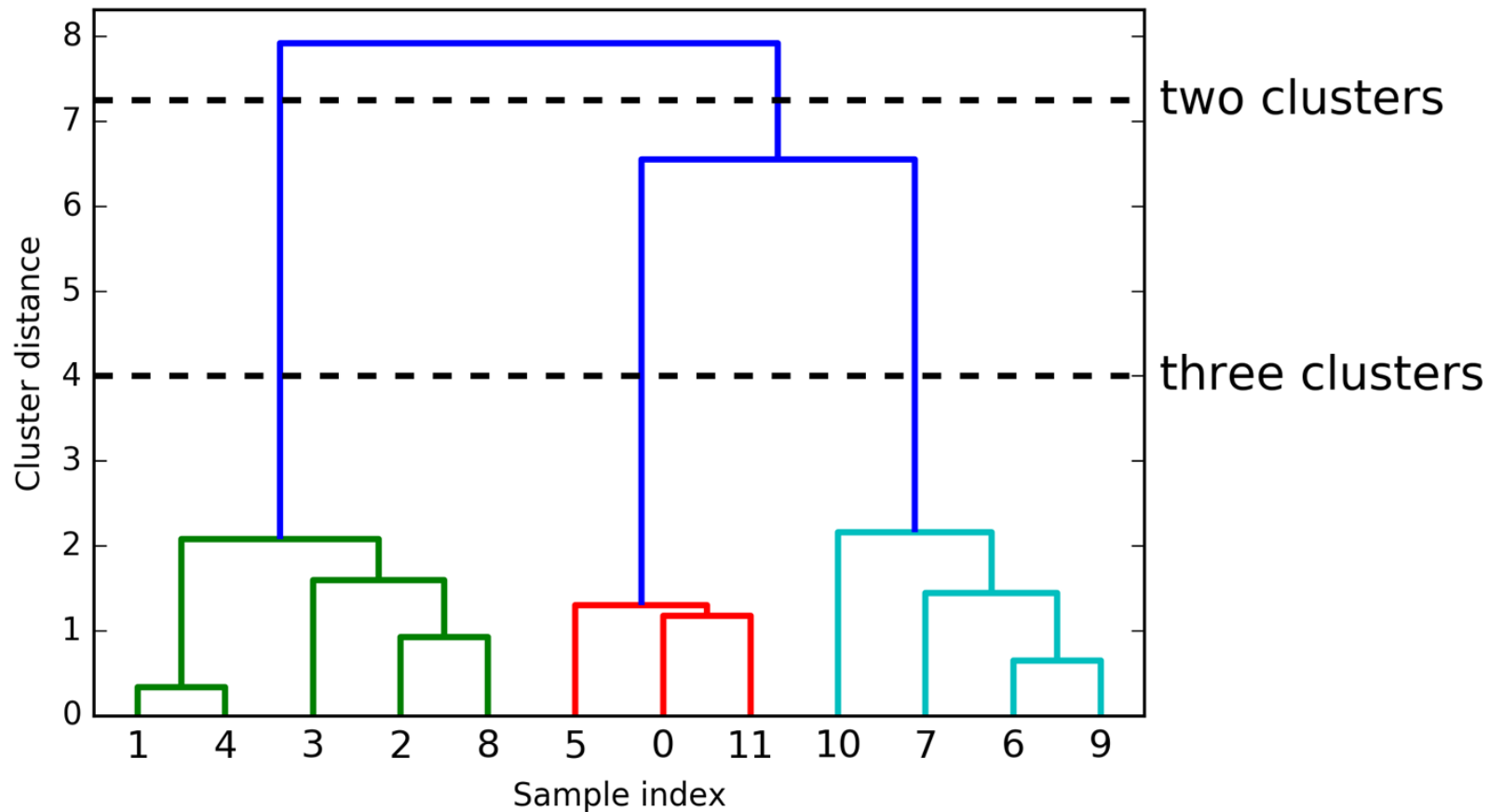


The y-axis in the dendrogram doesn't just specify when in the agglomerative algorithm two clusters get merged. The length of each branch also shows how far apart the merged clusters are.

The longest branches in this dendrogram are the three lines that are marked by the dashed line labeled "three clusters."

Dendrogram of the clustering shown with lines indicating splits into two and three clusters

Hierarchical clustering and dendrograms



That these are the longest branches indicates that going from three to two clusters meant merging some very far-apart points. We see this again at the top of the chart, where merging the two remaining clusters into a single cluster again bridges a relatively large distance.

Unfortunately, agglomerative clustering still fails at separating complex shapes.

Dendrogram of the clustering shown with lines indicating splits into two and three clusters

References

Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking by Foster Provost and Tom Fawcett

Introduction to Machine Learning with Python, a Guide for Data Scientists by Andreas C. Müller & Sarah Guido